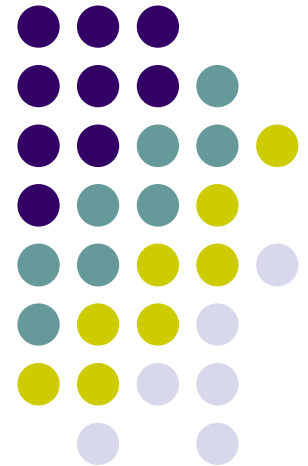
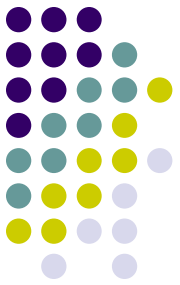


第3章 动态规划

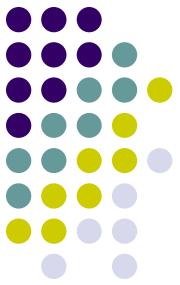
History does not occur again





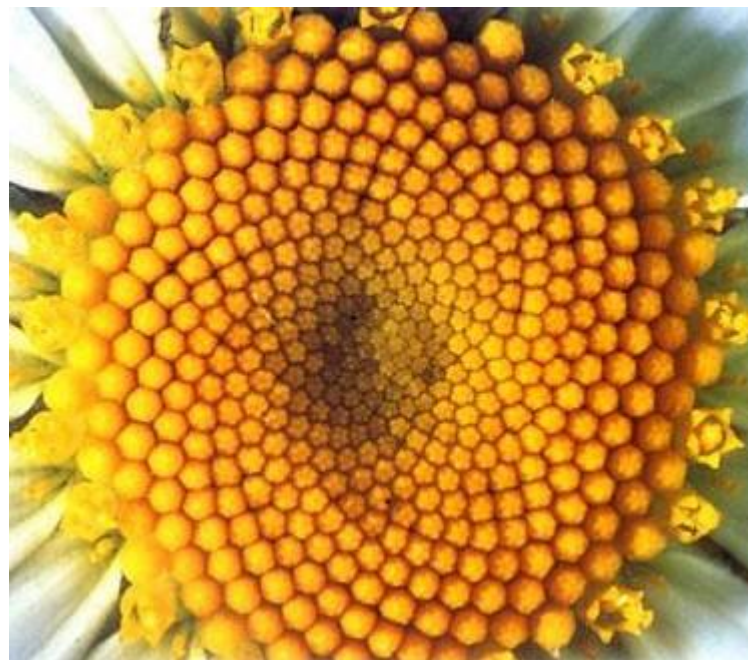
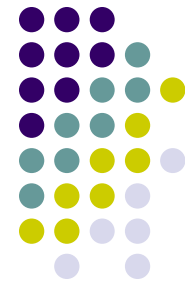
学习要点:

- 理解动态规划算法的概念
- 掌握动态规划算法的基本要素
 - (1) 最优子结构性质
 - (2) 重叠子问题性质
- 掌握设计动态规划算法的步骤
 - (1) 找出最优解的性质，并刻画其结构特征
 - (2) 递归地定义最优值
 - (3) 以自底向上的方式计算出最优值
 - (4) 根据计算最优值时得到的信息，构造最优解

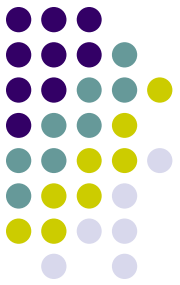


- 通过应用范例学习动态规划算法设计策略
 - (1) 矩阵连乘问题
 - (2) 最长公共子序列
 - (3) 最大子段和
 - (4) 凸多边形最优三角剖分
 - (5) 多边形游戏
 - (6) 图像压缩
 - (7) 电路布线
 - (8) 流水作业调度
 - (9) 背包问题
 - (10) 最优二叉搜索树

自然界中的斐波那契



Fibonacci number $F(n)$



$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

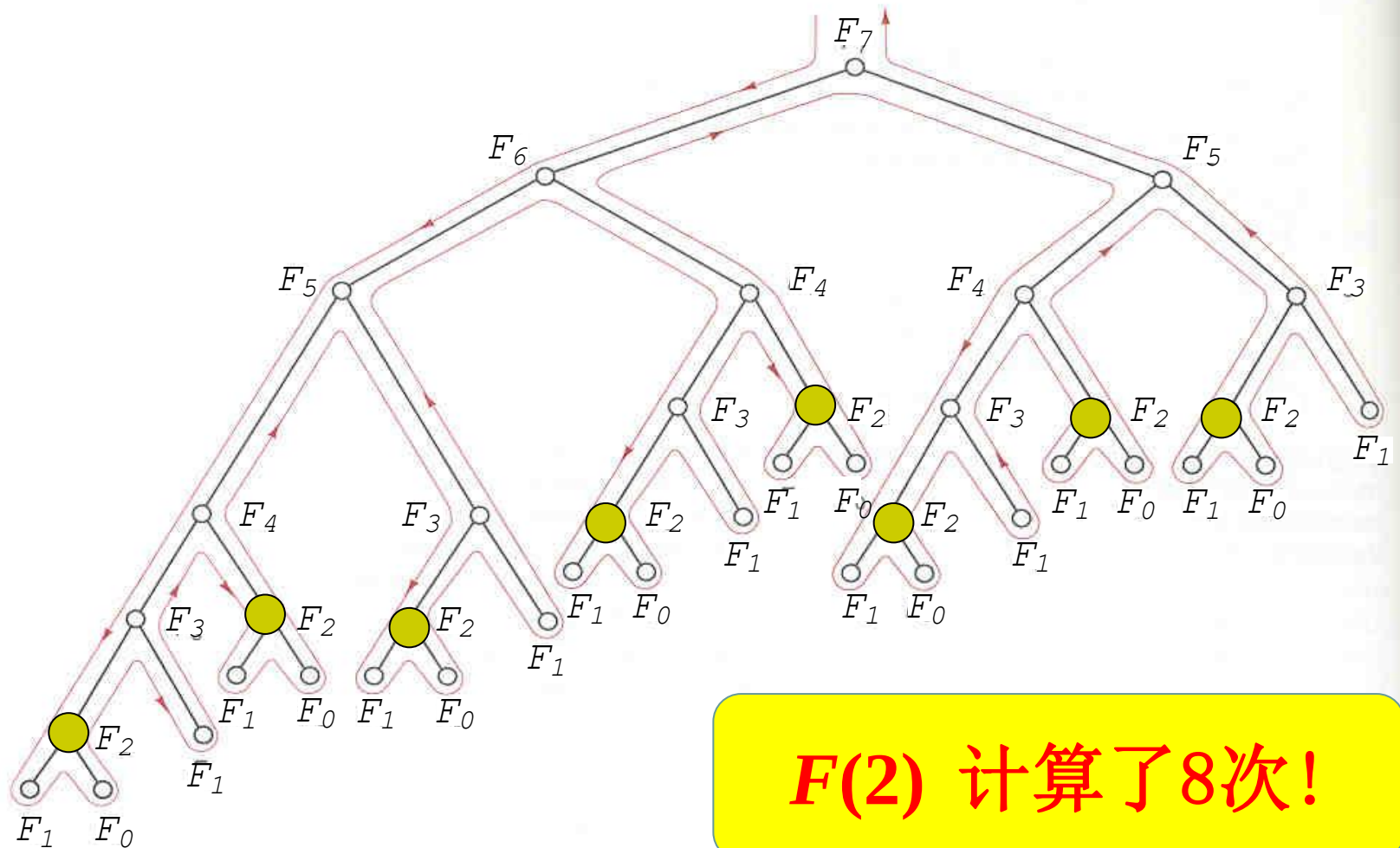
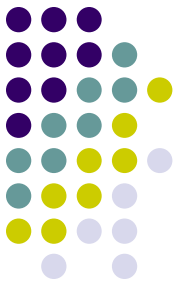
n	0	1	2	3	4	5	6	7	8	9	10
$F(n)$	1	1	2	3	5	8	13	21	34	55	89

Pseudo code for the recursive algorithm:

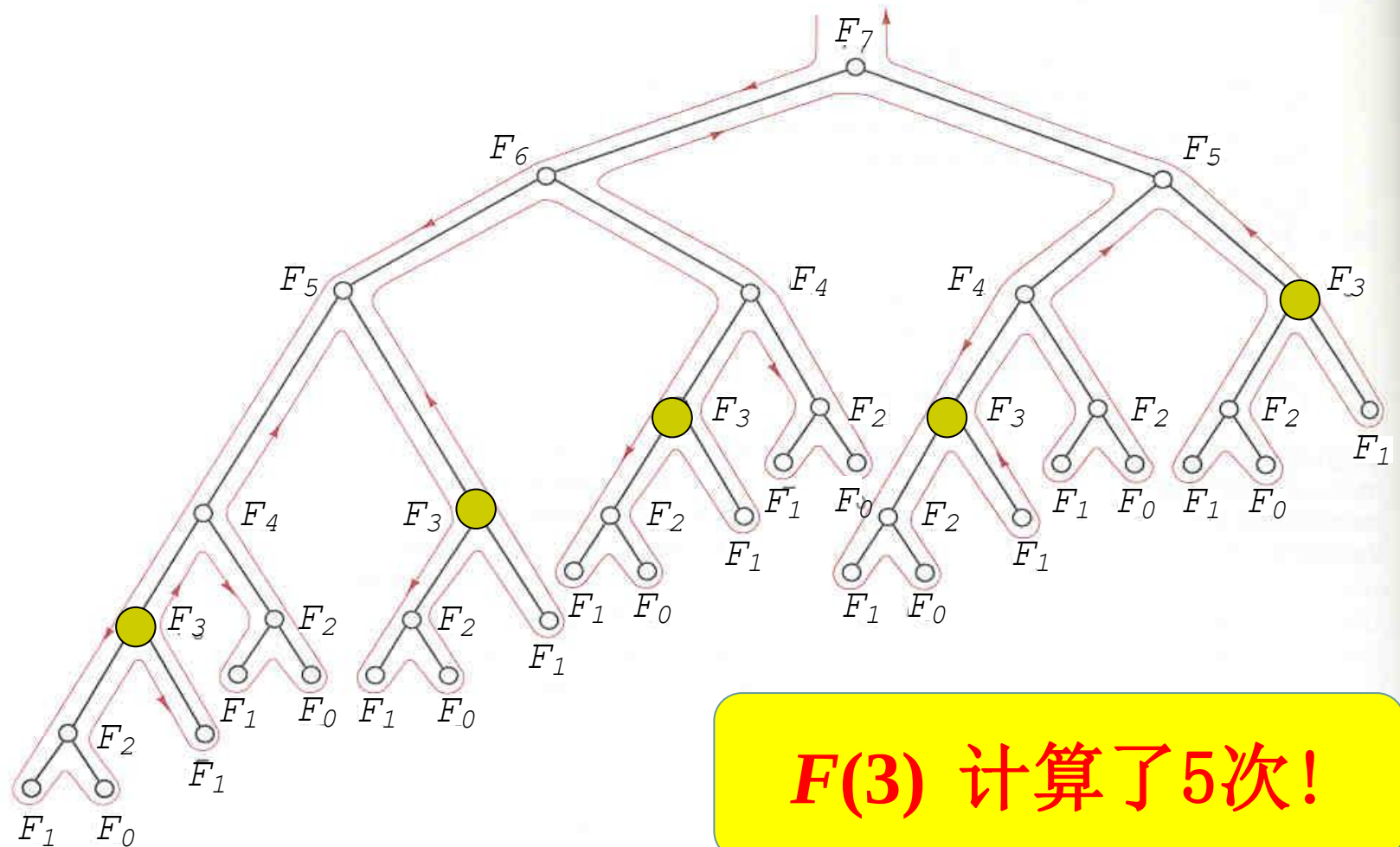
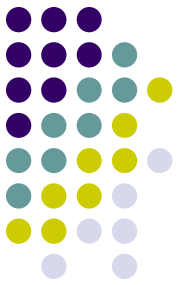
$F(n)$

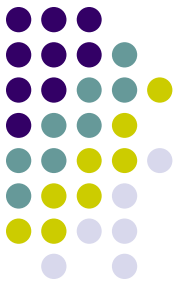
```
1  if  $n=0$  or  $n=1$  then
2      return 1
3  else
4      return  $F(n-1) + F(n-2)$ 
```

The execution of $F(7)$



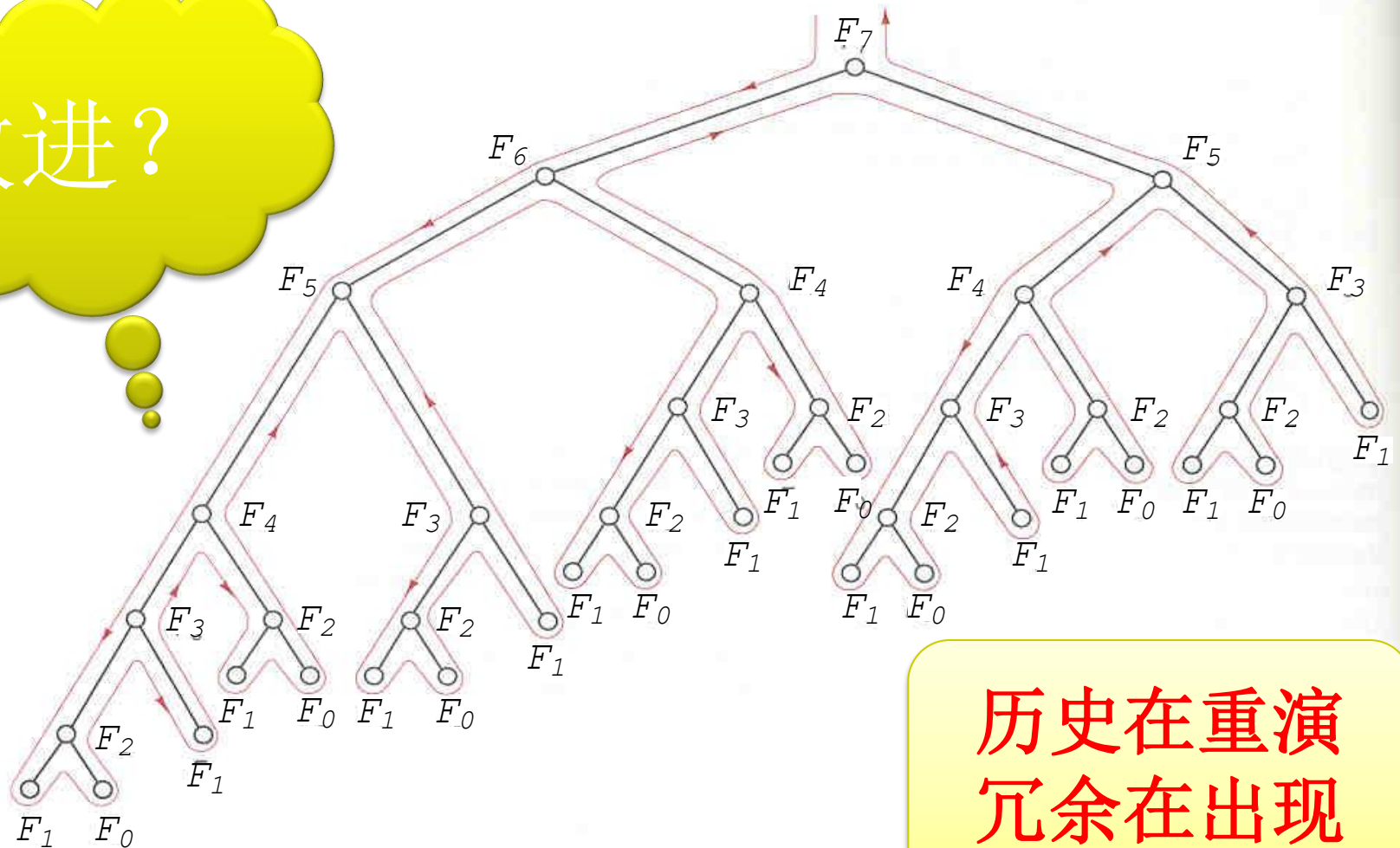
The execution of $F(7)$





The execution of $F(7)$

改进?



历史在重演
冗余在出现



Idea for improvement

- 保存计算结果 $F(i)$
- 再次调用时，先查表
 - 有结果，直接调用
 - 没结果，进入递归计算

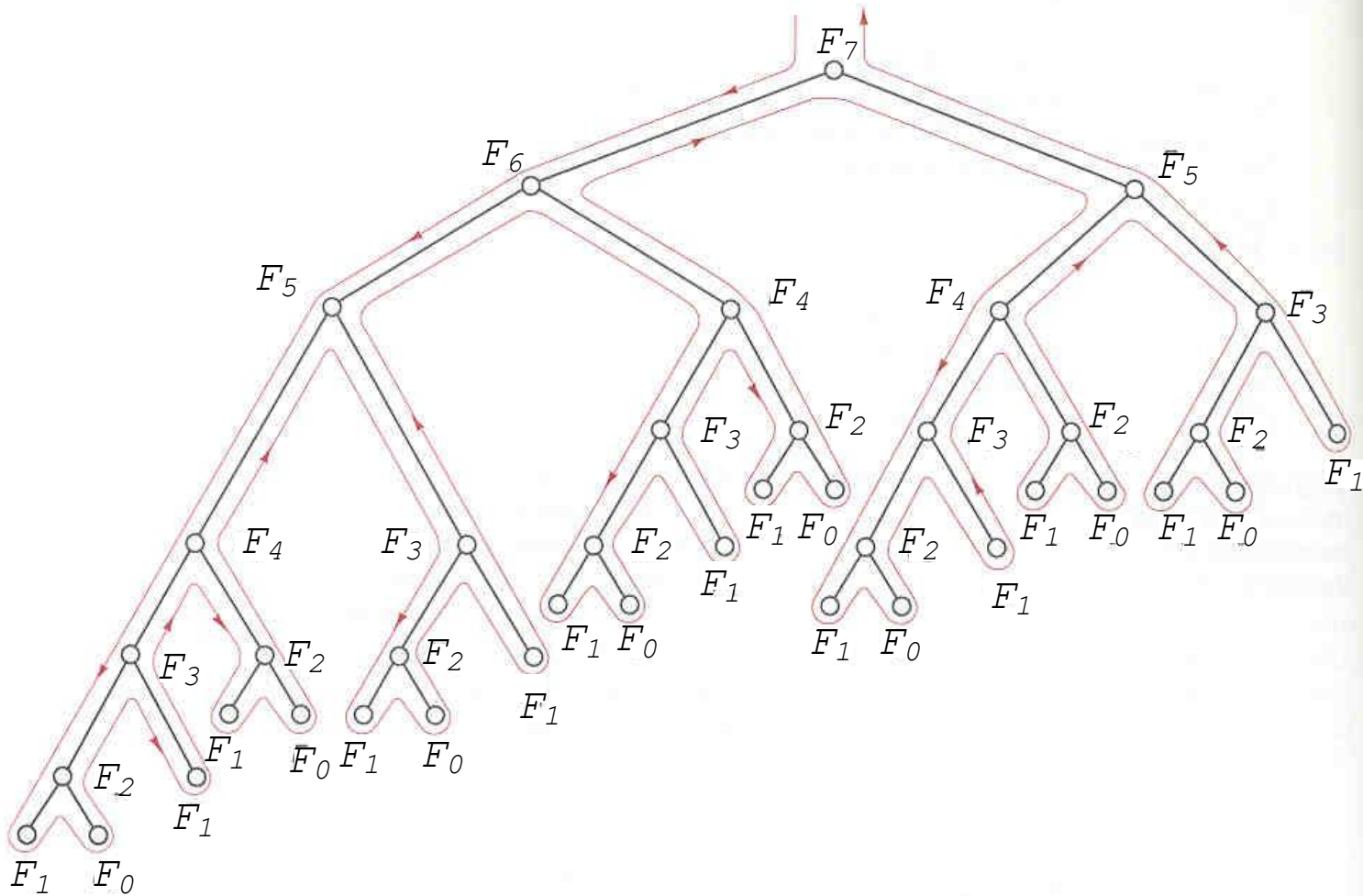
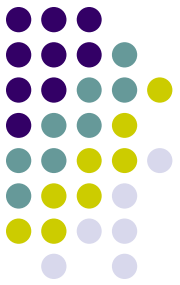
$F(n)$

```
1 if ( $v[n] < 0$ ) then
2    $v[n] \leftarrow F(n-1) + F(n-2)$ 
3 return  $v[n]$ 
```

Main()

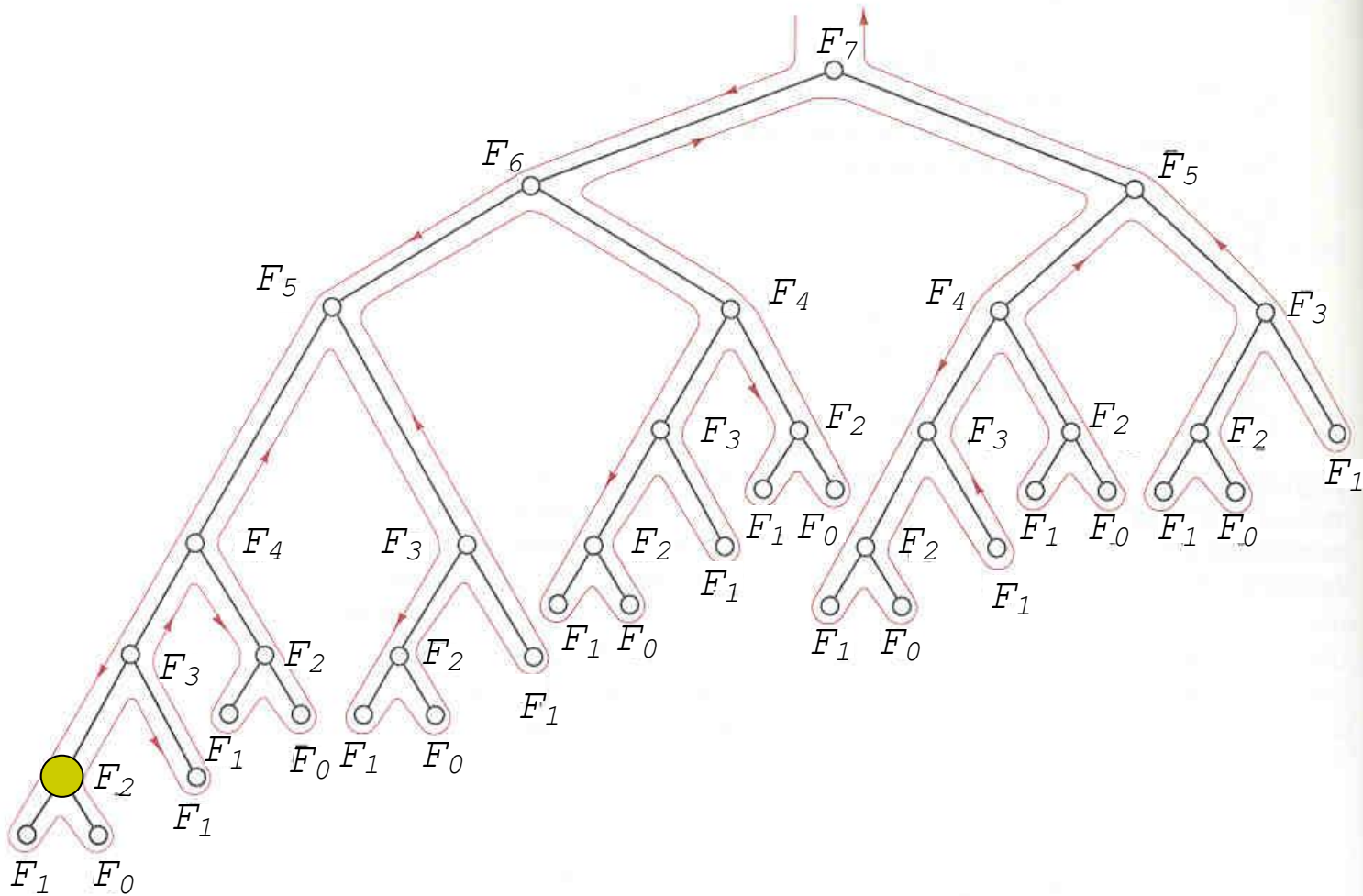
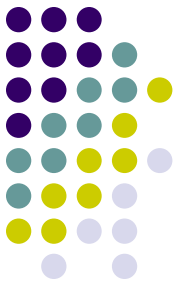
```
1  $v[0] = v[1] \leftarrow 1$ 
2 for  $i \leftarrow 2$  to  $n$  do
3    $v[i] = -1$ 
4 output  $F(n)$ 
```

The execution of $F(7)$



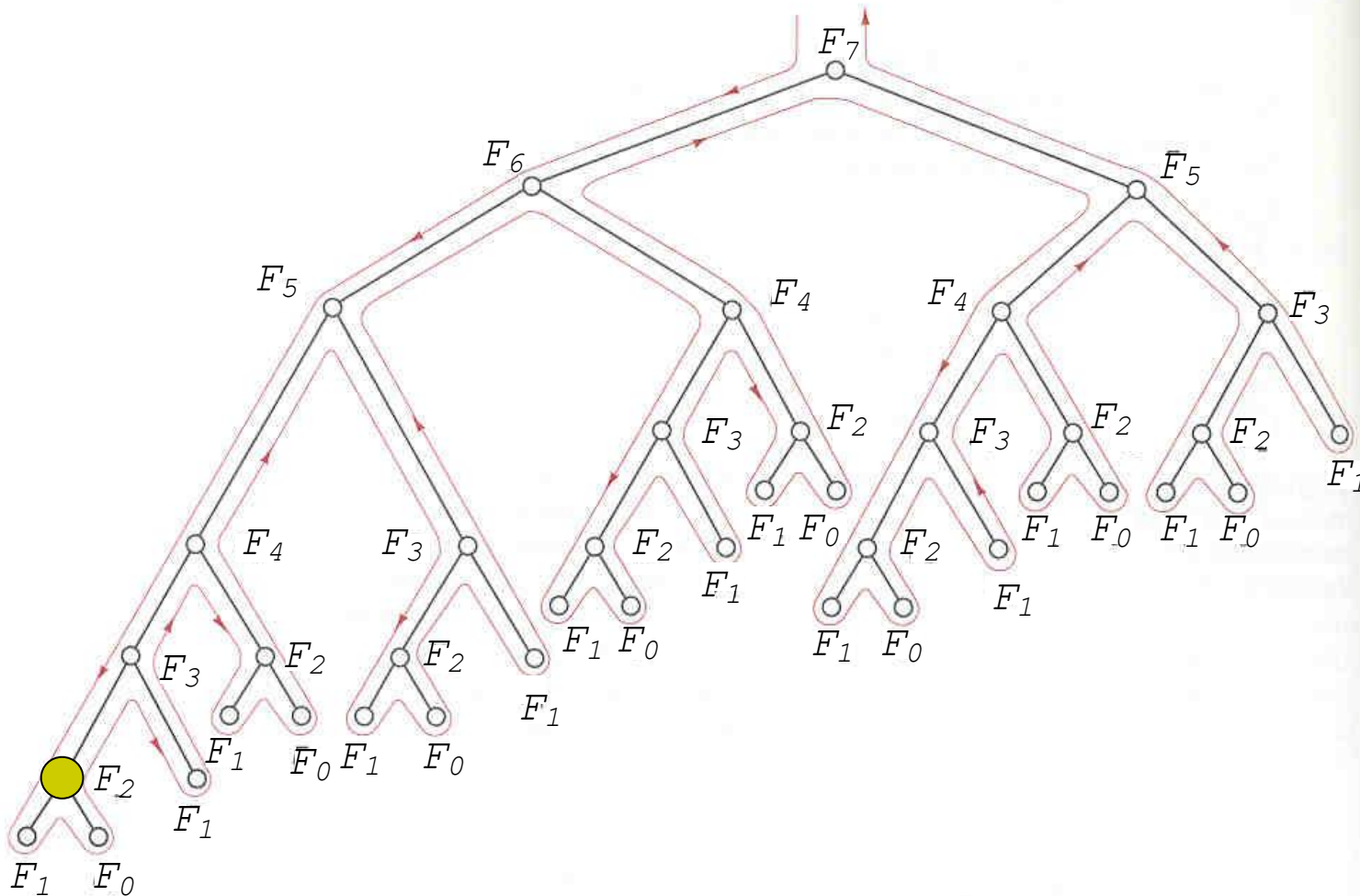
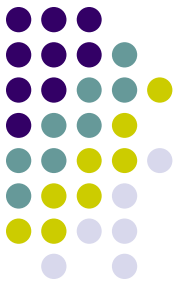
$v[0]$	1
$v[1]$	1
$v[2]$	-1
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

The execution of $F(7)$



$v[0]$	1
$v[1]$	1
$v[2]$	-1
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

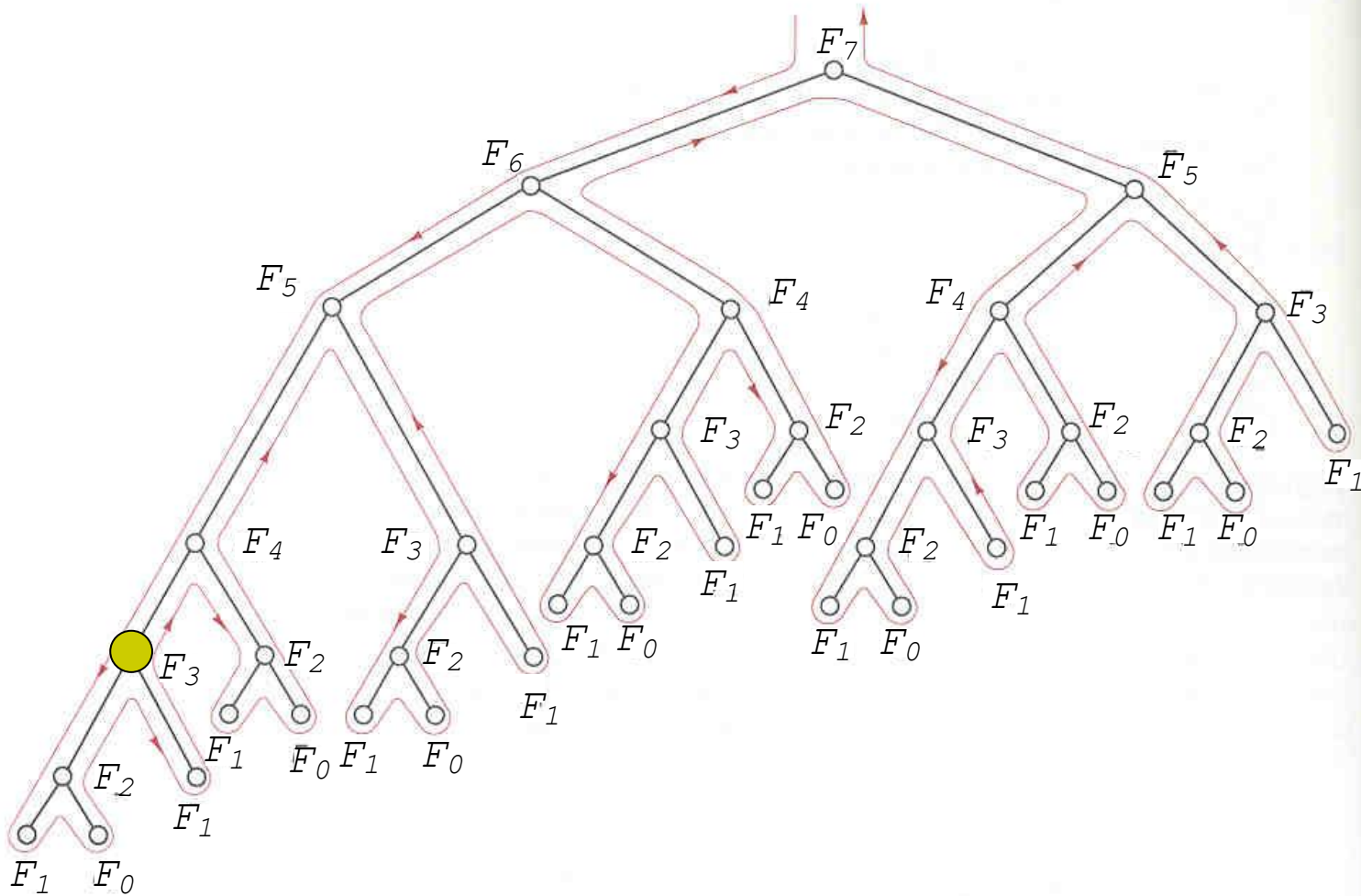
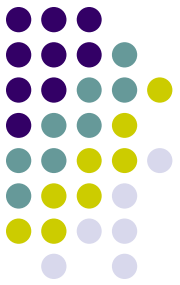
The execution of $F(7)$



$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

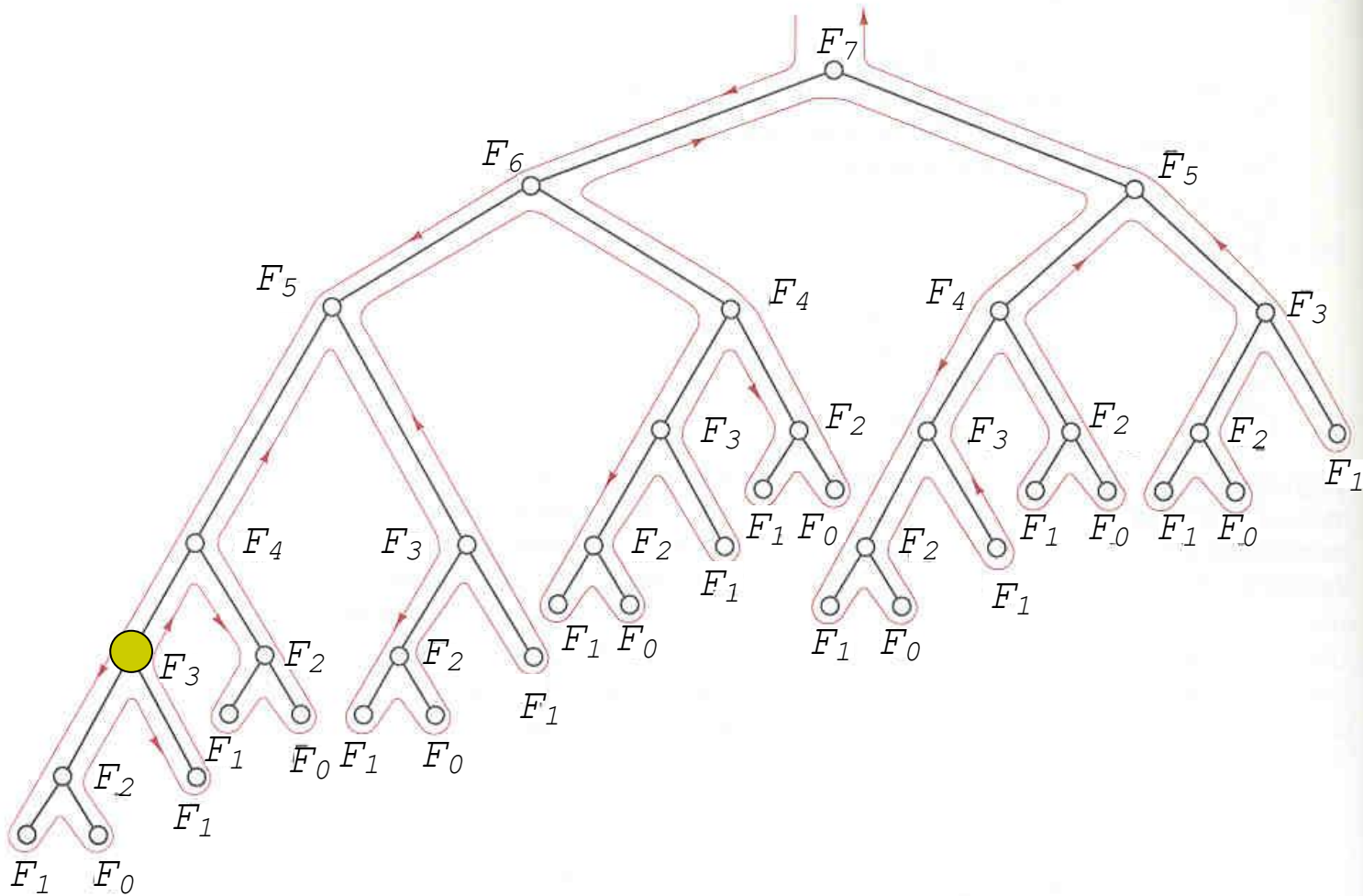
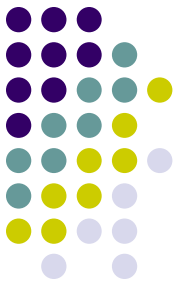
12

The execution of $F(7)$



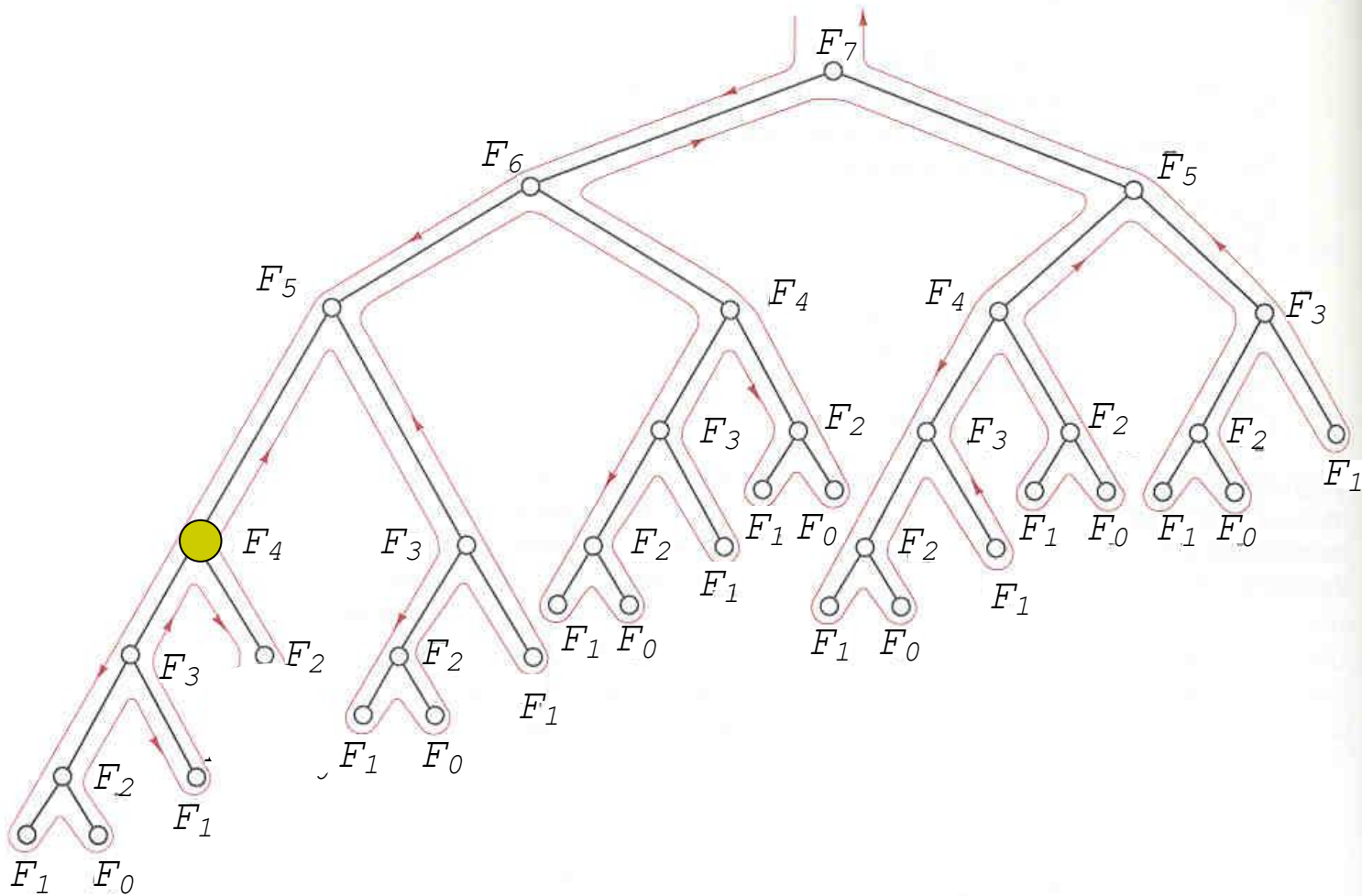
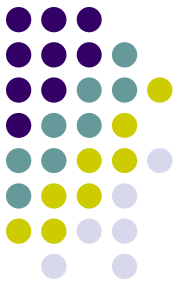
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

The execution of $F(7)$



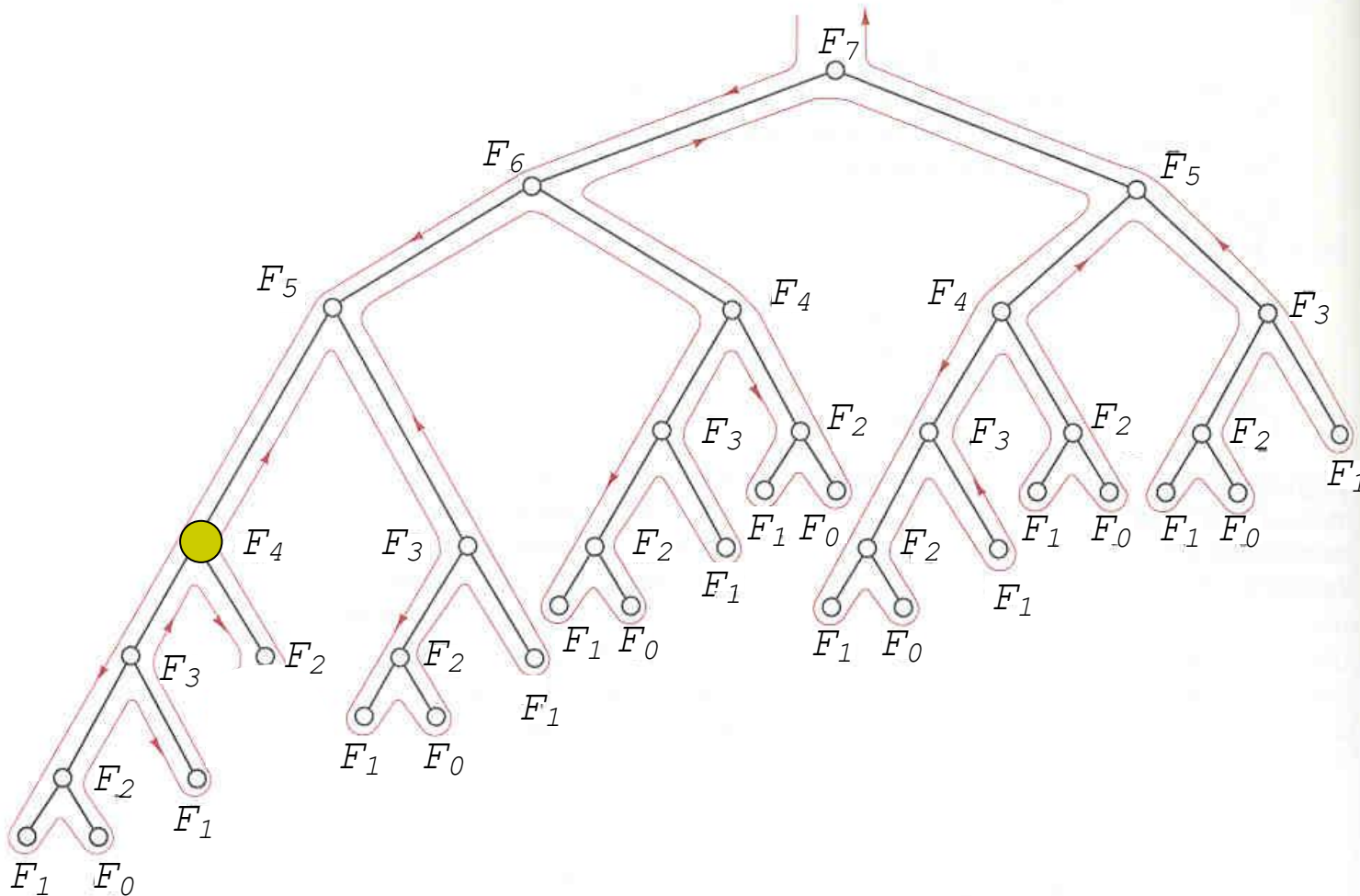
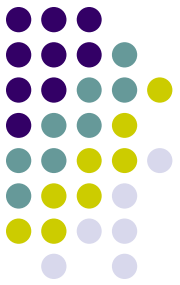
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

The execution of $F(7)$



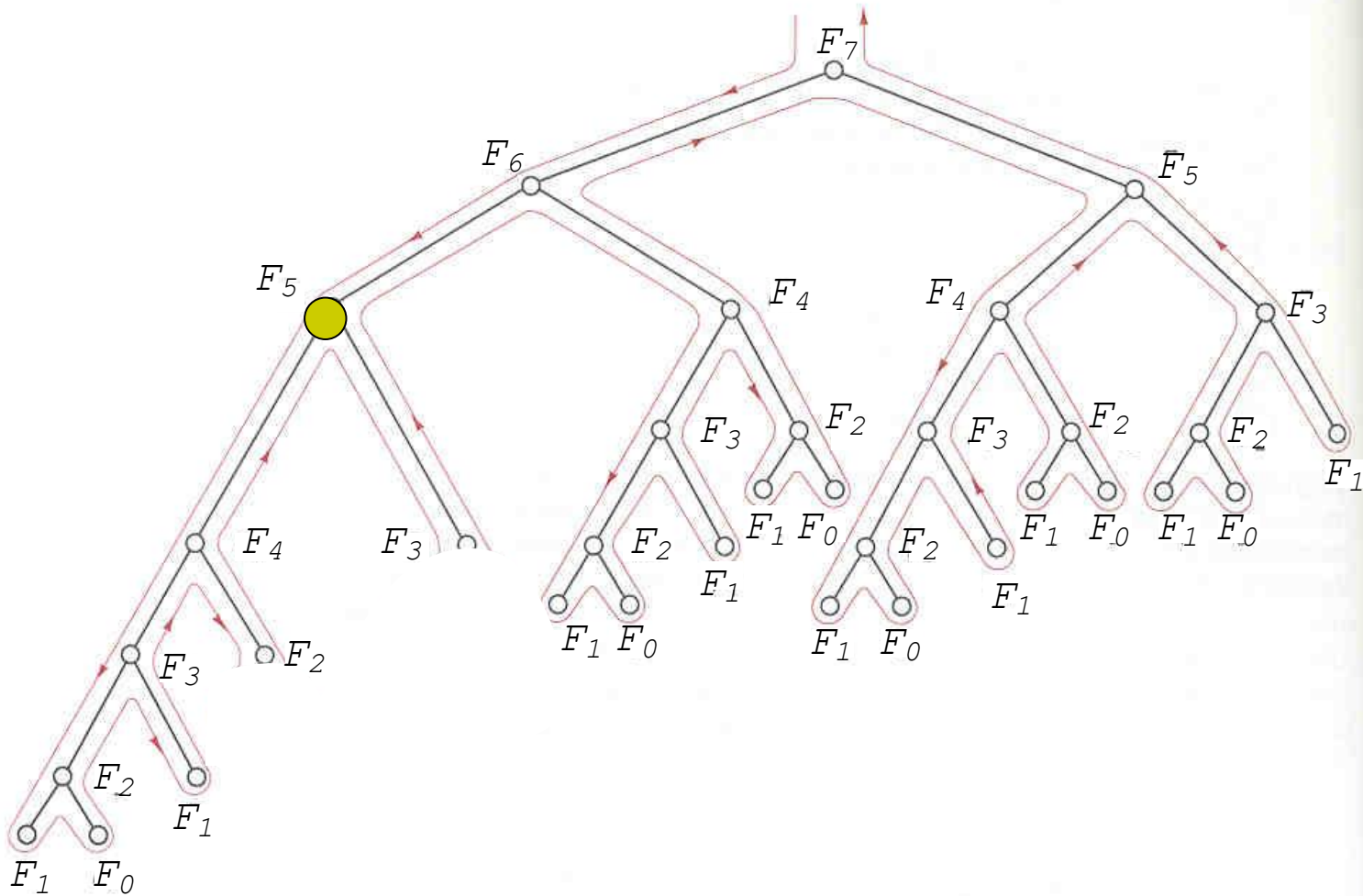
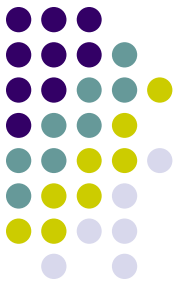
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

The execution of $F(7)$



$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

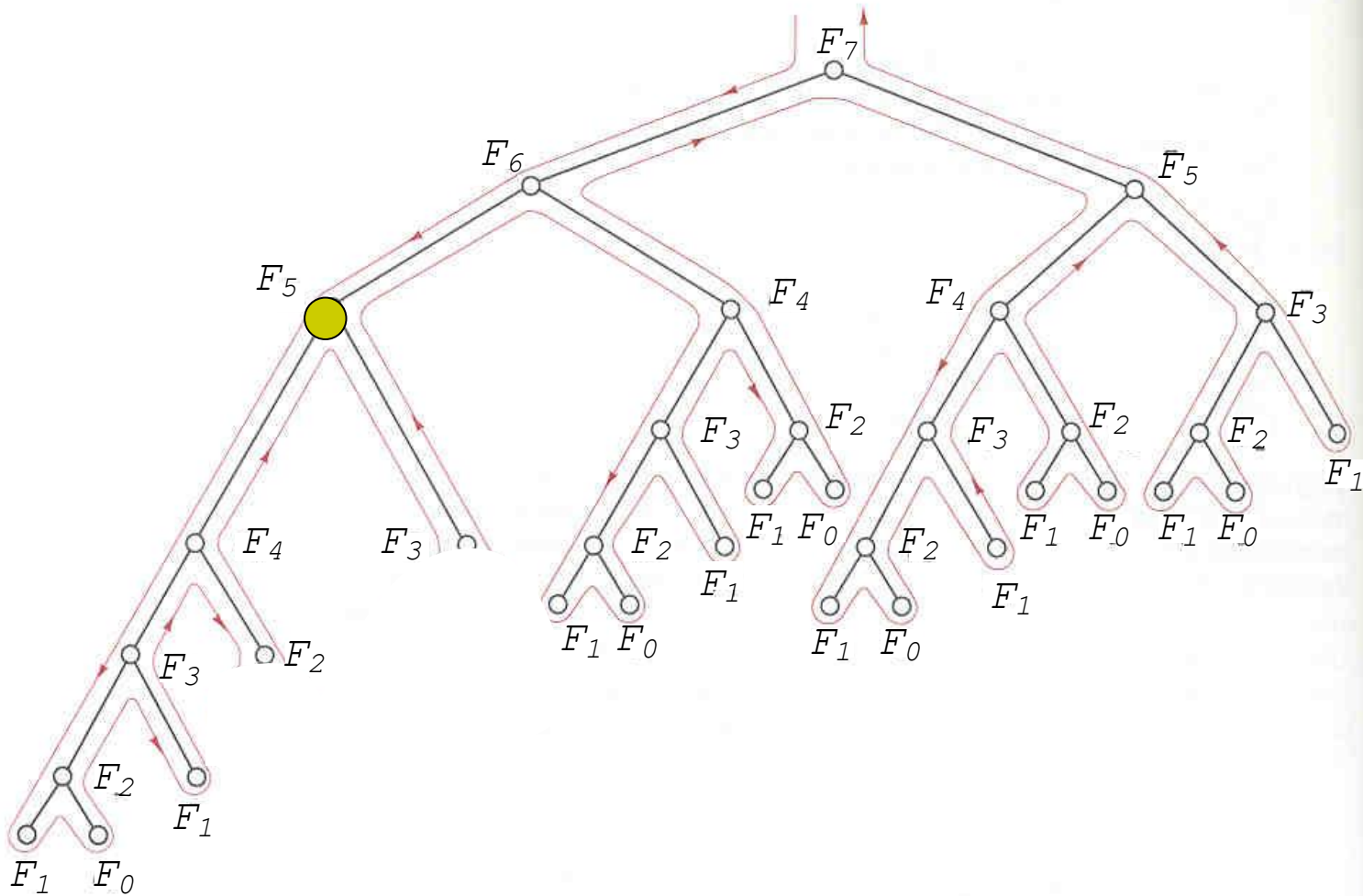
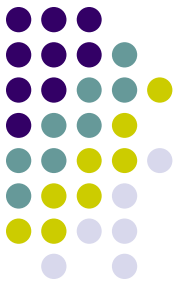
The execution of $F(7)$



$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

17

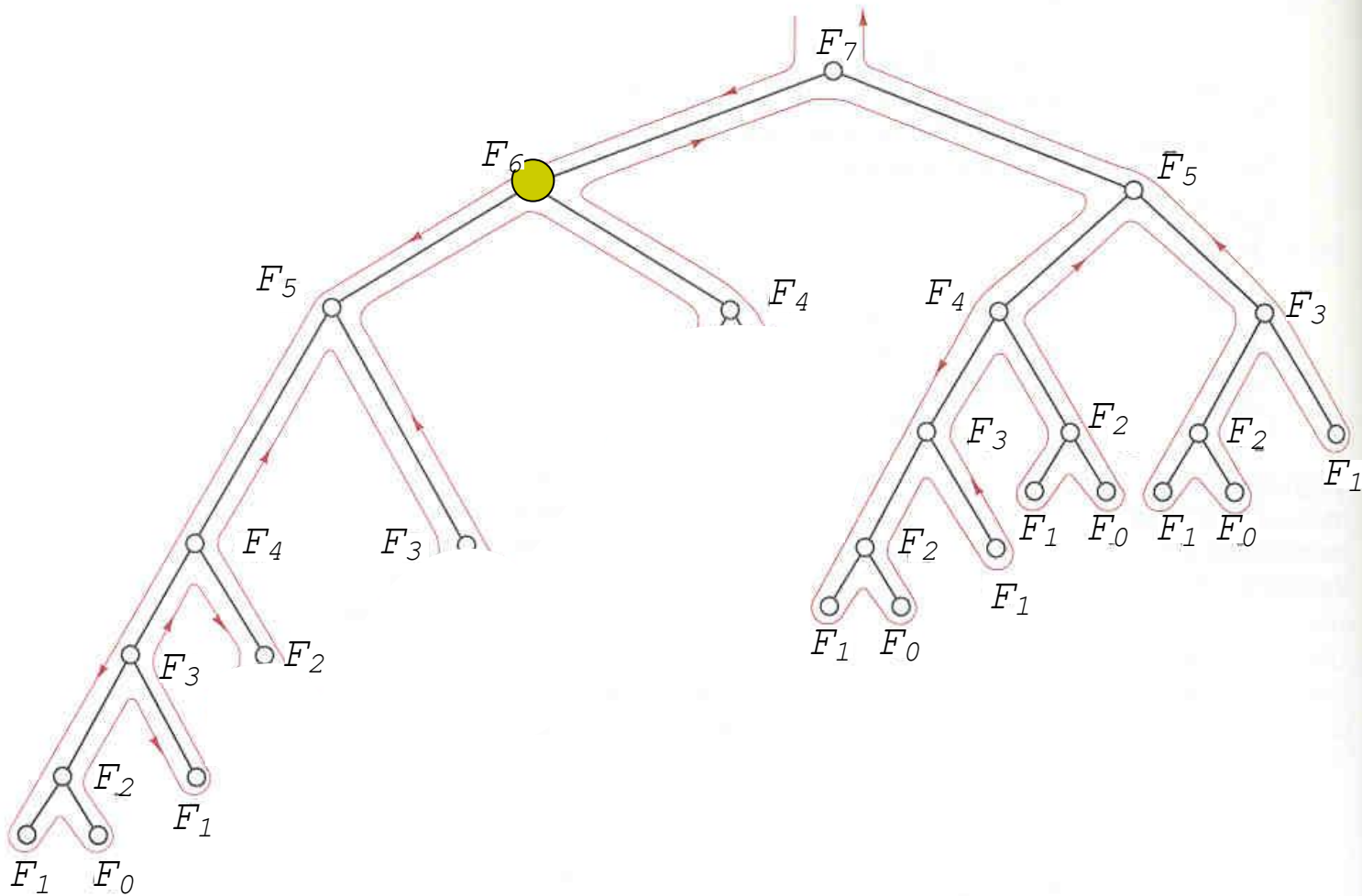
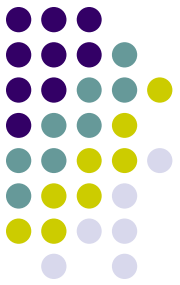
The execution of $F(7)$



$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	-1
$v[7]$	-1

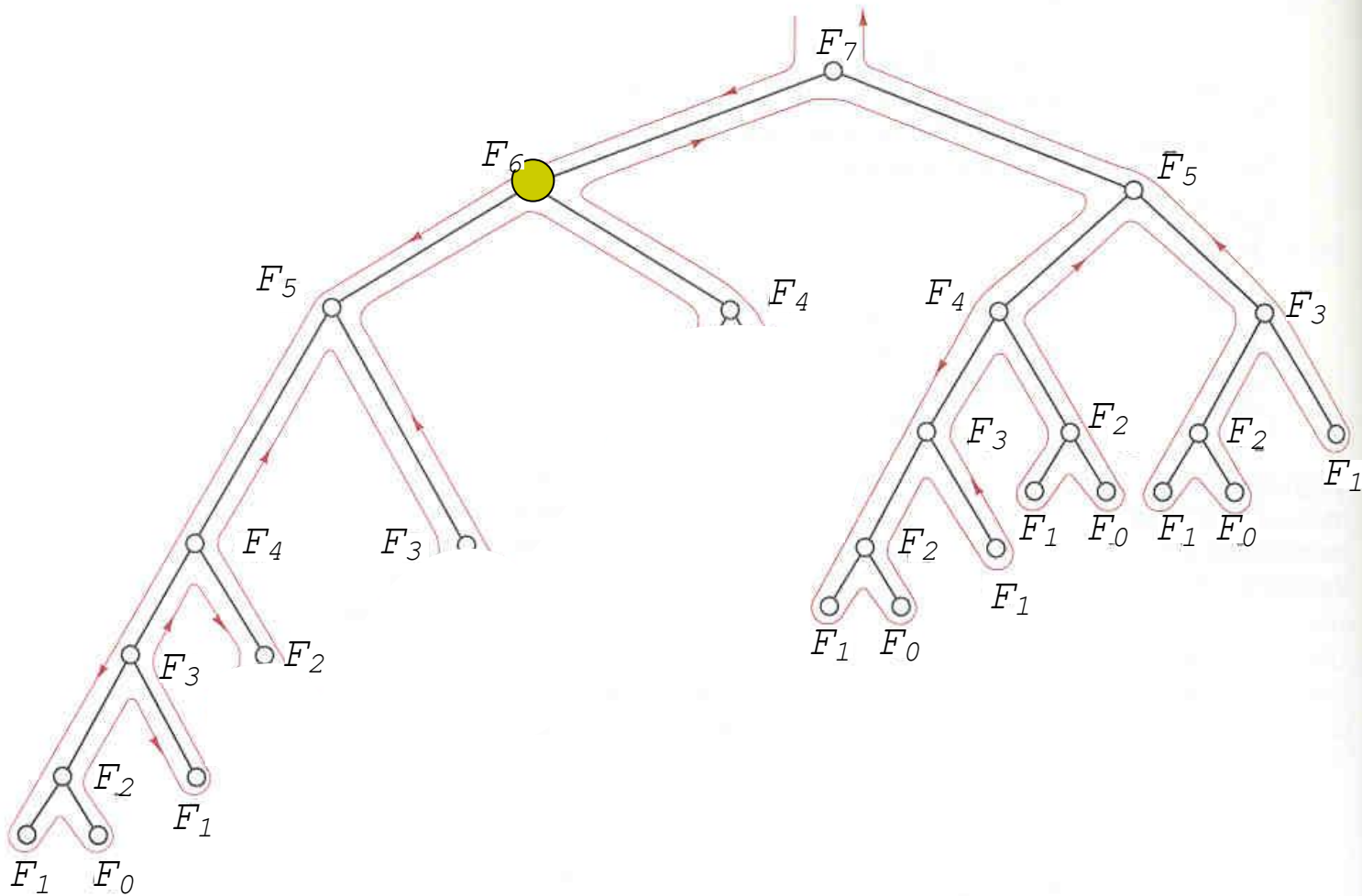
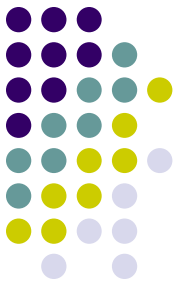
18

The execution of $F(7)$



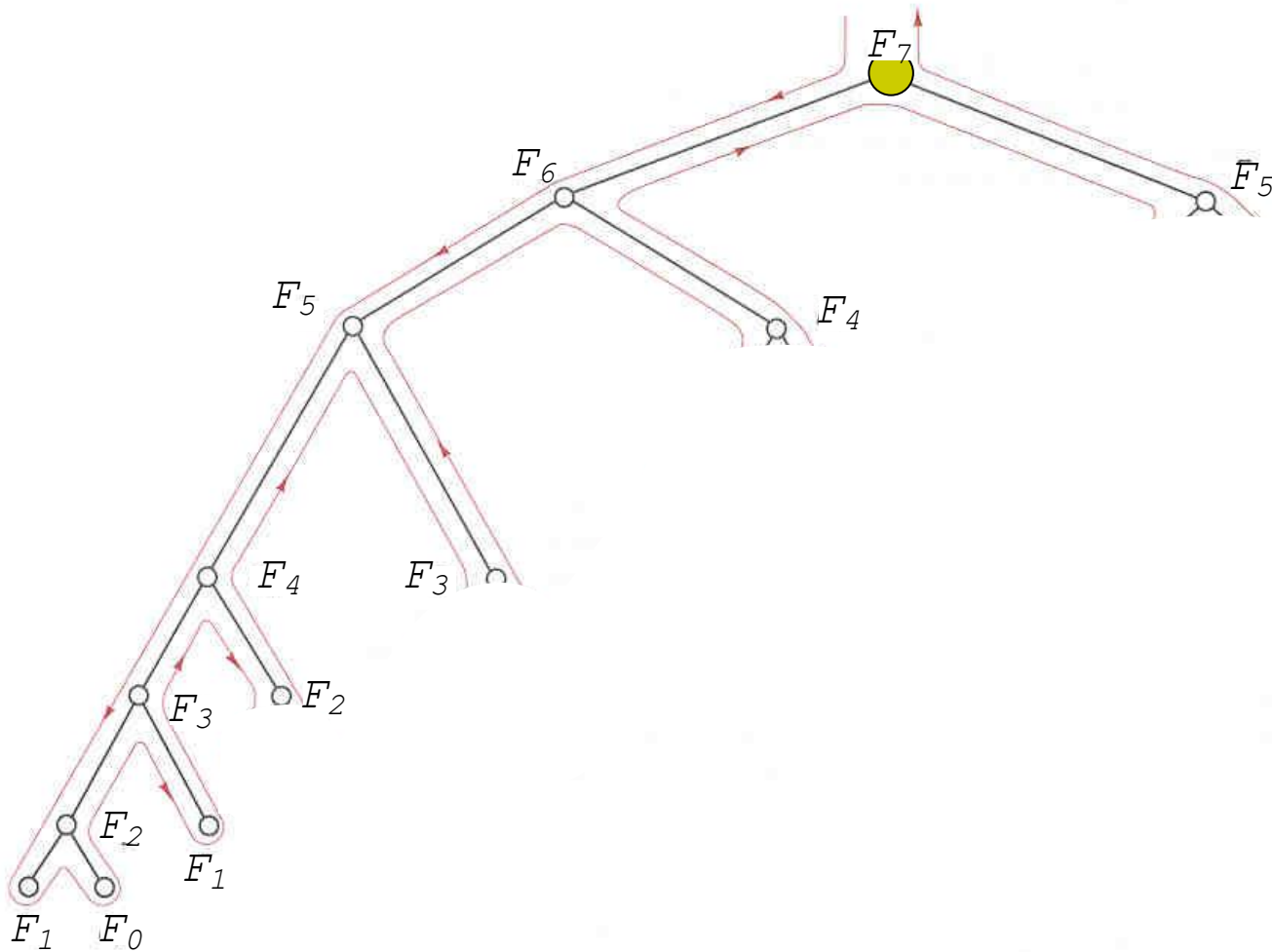
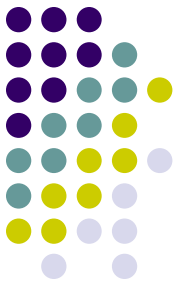
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	-1
$v[7]$	-1

The execution of $F(7)$



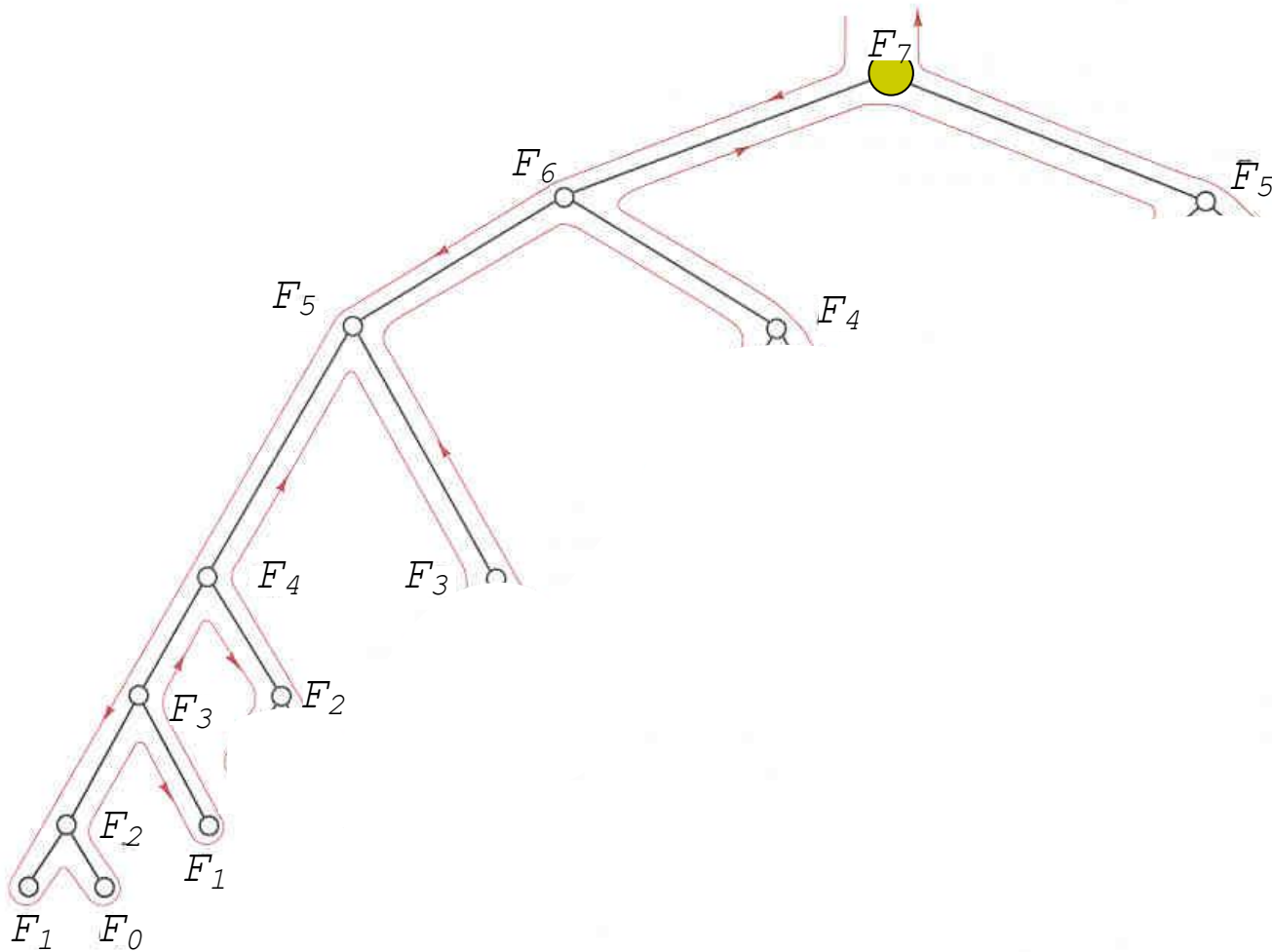
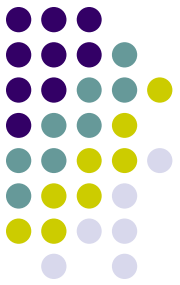
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	13
$v[7]$	-1

The execution of $F(7)$



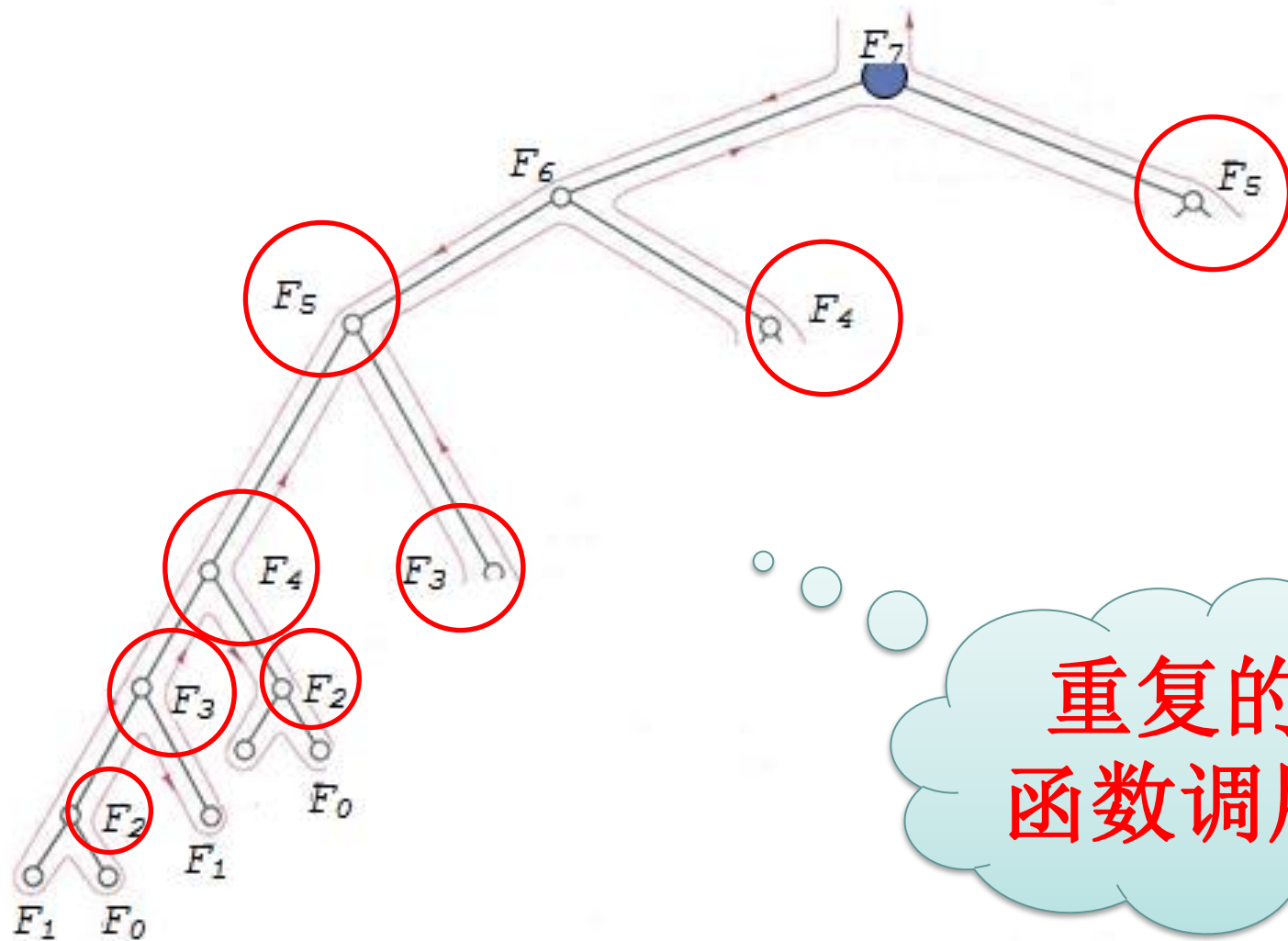
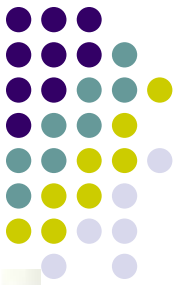
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	13
$v[7]$	-1

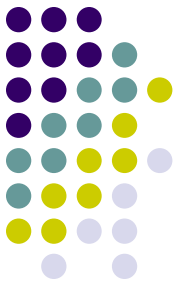
The execution of $F(7)$



$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	13
$v[7]$	21

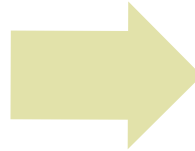
Can we do even better?





Can we do even better?

自顶向下
Top-down



自底向上
Bottom-up

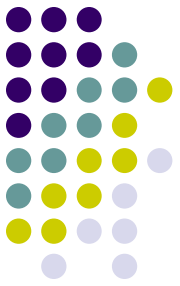
- 递归

- 递推

递推方式
节约大量无用
递归调用

```
F(n)
1   A[0] = A[1] ← 1
2   for i ← 2 to n do
3       A[i] ← A[i-1] + A[i-2]
4   return A[n]
```

分治递归 vs 动态规划



分治递归:

```
F(n)
1  if n=0 or n=1 then
2      return 1
3  else
4      return F(n-1) + F(n-2)
```

效率低!

动态规划:

```
F(n)
1  A[0] = A[1] ← 1
2  for i ← 2 to n do
3      A[i] ← A[i-1] + A[i-2]
4  return A[n]
```

高效!
时间复杂度 $O(n)$

Summary of the methodology



- Write down a formula that relates a solution of a problem with those of sub-problems.
E.g. $F(n) = F(n-1) + F(n-2)$.
- **Index** the sub-problems so that they can be **stored** and **retrieved** easily in a table (i.e., array)
- Fill the table in some **bottom-up** manner; start filling the solution of the smallest problem.
 - This ensures that when we solve a particular sub-problem, the solutions of all the smaller sub-

For historical reasons, we call such methodology
Dynamic Programming.

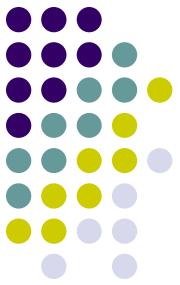
In the late 40's (when computers were rare),
programming refers to the "tabular method".



- **动态规划 (dynamic programming)** 是运筹学的一个分支, 20世纪50年代初美国数学家R.E.Bellman等人在研究**多阶段决策过程** (Multistep decision process) 的优化问题时, 提出了著名的**最优性原理**, 把多阶段过程转化为一系列单阶段问题, 逐个求解, 创立了解决这类过程优化问题的新方法——**动态规划**。
- **多阶段决策问题**: 求解的问题可以划分为一系列相互联系的阶段, 在每个阶段都需要做出决策, 且一个阶段决策的选择会影响下一个阶段的决策, 从而影响整个过程的活动路线, 求解的目标是选择各个阶段的决策是整个过程达到**最优**。

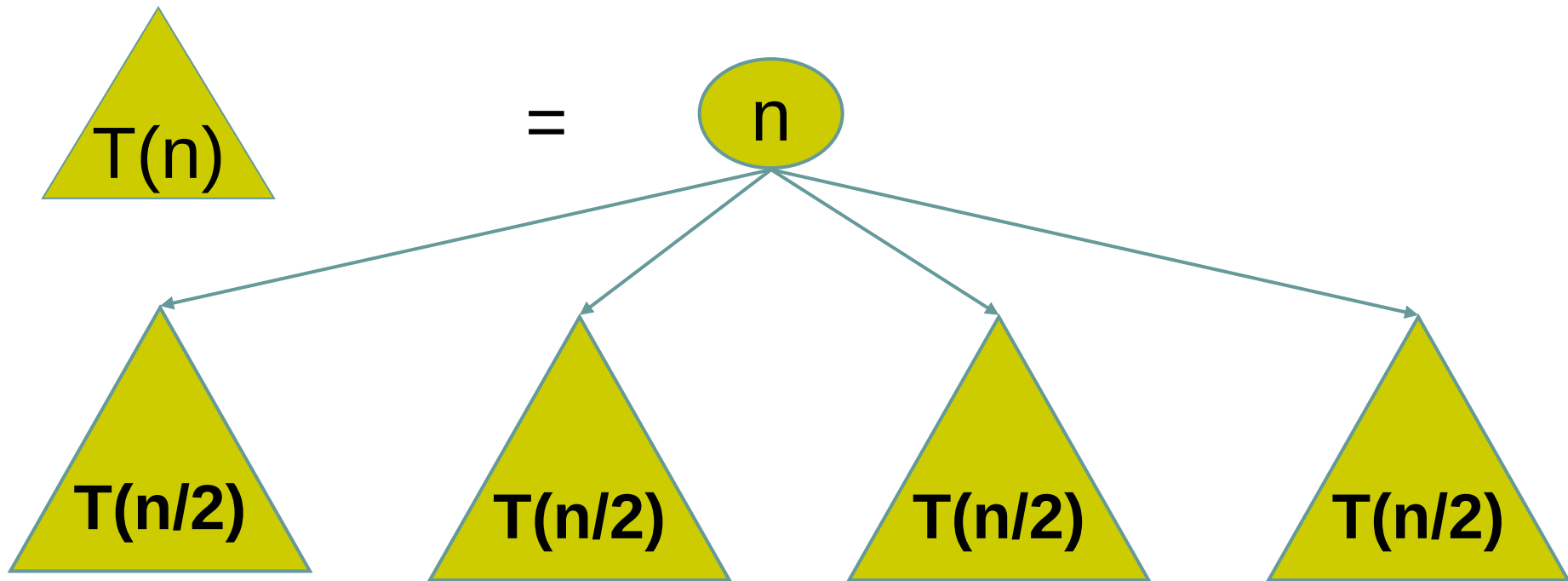


- 动态规划主要用于求解以时间划分阶段的动态过程的优化问题，但是是一些与时间无关的静态规划（如线性规划、非线性规划），可以人为地引进时间因素，把它视为多阶段决策过程，也可以用动态规划方法方便地求解。
- 动态规划是考察问题的一种途径，或是求解某类问题的一种方法。
- 动态规划问世以来，在经济管理、生产调度、工程技术和最优控制等方面得到了广泛的应用。例如最短路线、库存管理、资源分配、设备更新、排序、装载等问题，用动态规划方法比其它方法求解更为方便。



算法总体思想

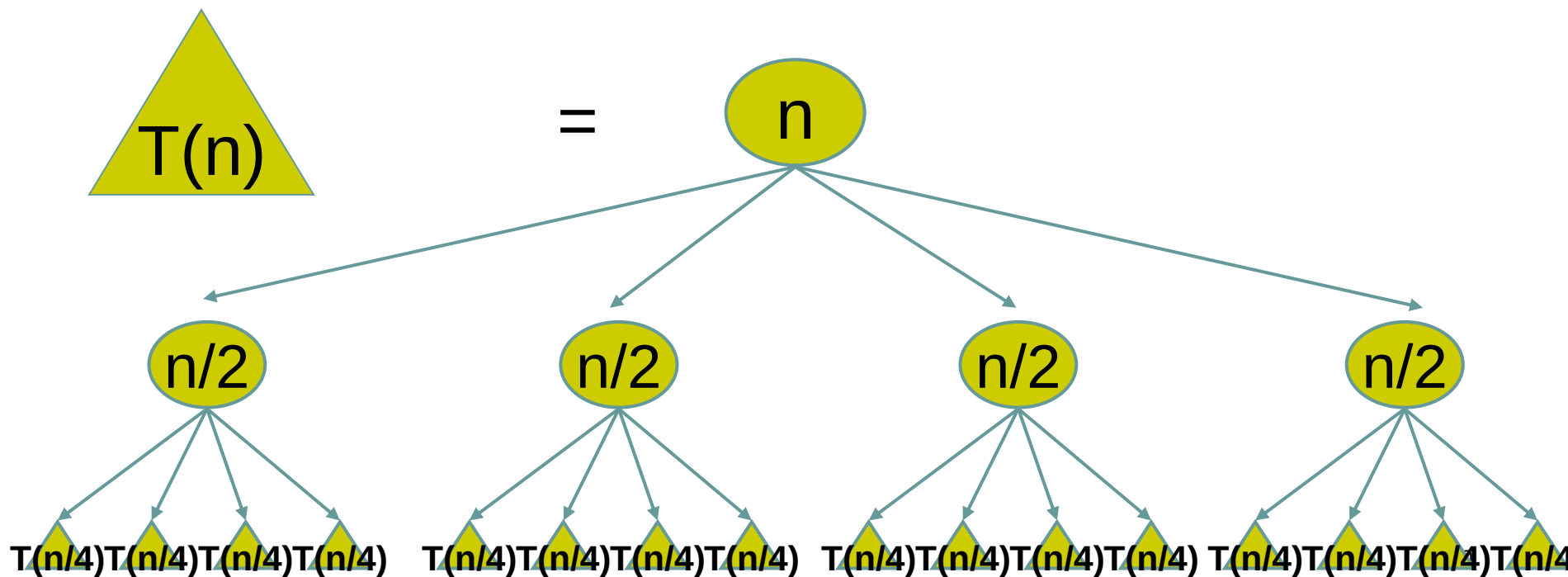
- 动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题，先求解子问题，再从子问题的解得到原问题的解。

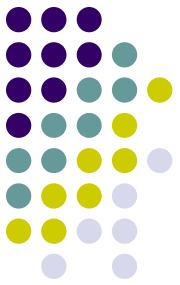




算法总体思想

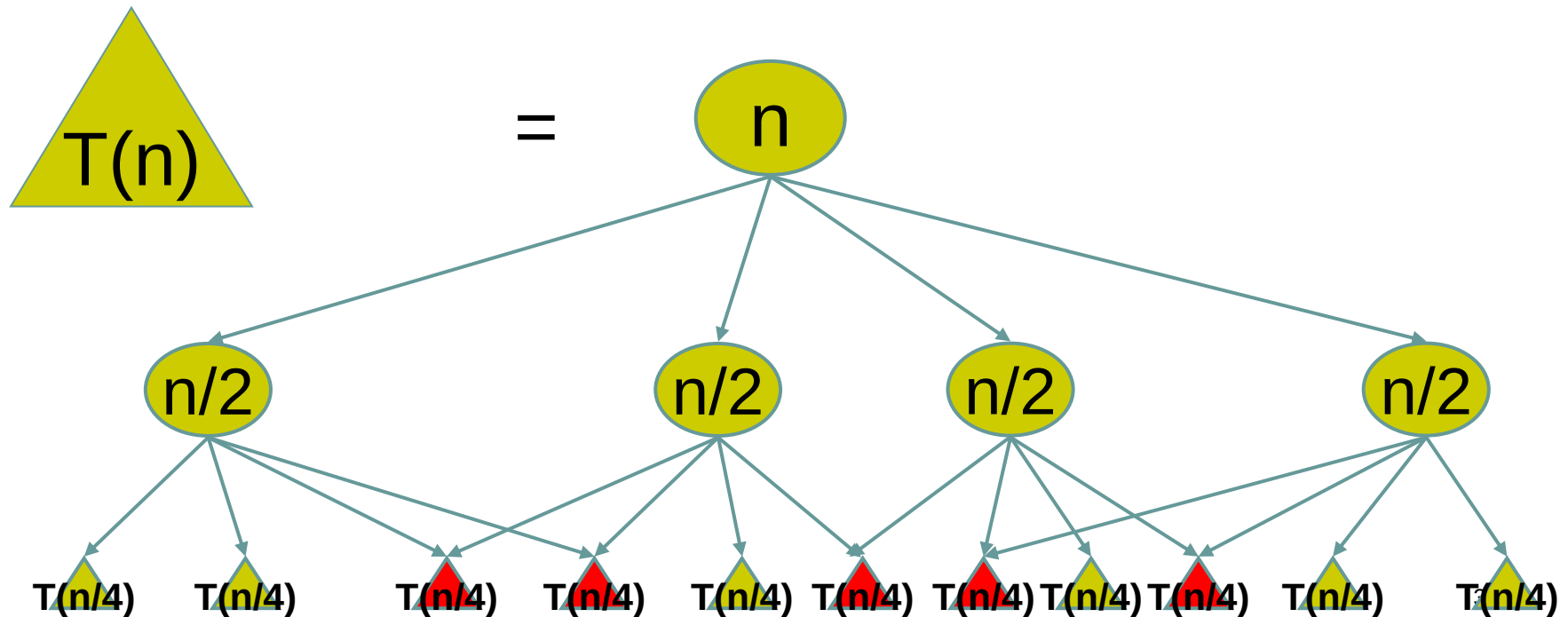
- 但是经分解得到的子问题往往不是互相独立的。不同子问题的数目常常只有多项式量级。在用分治法求解时，有些子问题被重复计算了许多次。





算法总体思想

- 保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。
- 动态规划法用一个表来记录所有已解决的子问题的答案。具体的动态规划算法尽管多种多样，但它们具有相同的填表方式。



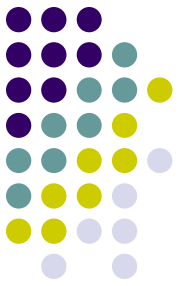


回顾分治法

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有**最优子结构性质**
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用**动态规划**较好。



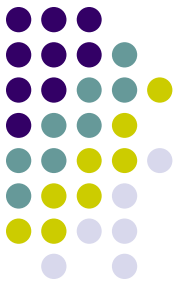
动态规划基本步骤

找出最优解的性质
并刻画其结构特征

递归地定义最优值

以自底向上的方式计算出最优值

根据计算最优值时得到的信息
构造最优解

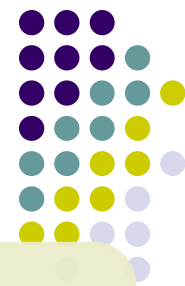


动态规划基本步骤

1. 找出最优解的性质，并刻画其结构特征。
2. 递归地定义最优值。
3. 以自底向上的方式计算出最优值。
4. 根据计算最优值时得到的信息，构造最优解。

🌿 步骤①~③是动态规划算法的基本步骤。如果只需求出最优值的情形，步骤④可以省略

🌿 若需求出问题的一个最优解，则必须执行步骤④，步骤③中记录的信息是构造最优解的基础；



分治递归

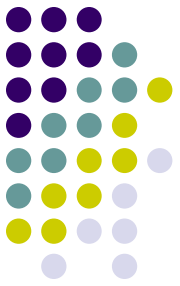
- 递归与分治策略
- Top-down

动态规划

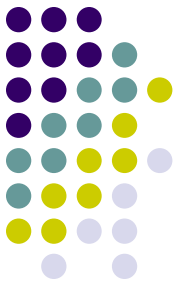
- 查表方式
- 最优子结构
- Bottom-up

贪心算法

- 贪心性质
- Top-down

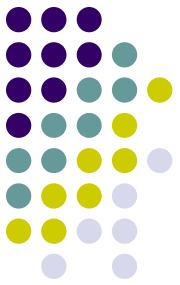


矩阵连乘问题



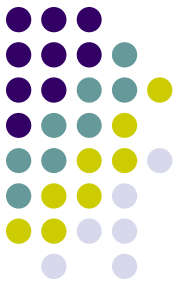
矩阵连乘问题

问题描述： 给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，
其中 A_i 与 A_{i+1} 是可乘的， $i=1, 2, \dots, n-1$ 。
如何确定矩阵连乘积的计算次序，使得
依此次序计算矩阵连乘积所需要的数乘
次数最少。



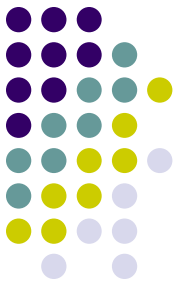
矩阵连乘问题

- 矩阵乘法满足结合律
 - ➔ 矩阵连乘可以有不同的计算次序。
- 矩阵连乘的计算次序可以用加括号的方式来确定
 - ➔ 若矩阵连乘已完全加括号，则其计算次序完全确定。



矩阵连乘问题

- ◆ 完全加括号的矩阵连乘积可递归地定义为：
 - (1) 单个矩阵是完全加括号的；
 - (2) 矩阵连乘积 A 是完全加括号的，则 A 可表示为2个完全加括号的矩阵连乘积 B 和 C 的乘积并加括号，即 $A=(BC)$ 。



矩阵连乘问题

- ◆ 例，有四个矩阵**A**, **B**, **C**, **D**，它们的维数分别是：

$A=50 \times 10$, $B=10 \times 40$, $C=40 \times 30$, $D=30 \times 5$

连乘积**ABCD**共有五种完全加括号的方式

$(A((BC)D))$ **16000**

$(A(B(CD)))$ **10500**

$((AB)(CD))$ **36000**

$(((AB)C)D)$ **87500**

$((A(BC))D)$ **34500**



矩阵连乘问题

- **穷举法：**列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

➤ **复杂性分析：**

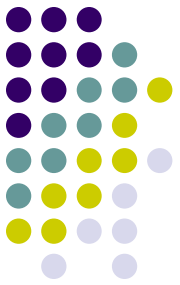
用 $p(n)$ 表示 n 个矩阵链乘的穷举法计算成本，如果将 n 个矩阵从第 k 和第 $k+1$ 处隔开，对两个子序列再分别加扩号，则可以得到下面递归式：

$$p(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} p(k)p(n-k) & n > 1 \end{cases}$$

$\Rightarrow p(n) = C(n-1)$ 为Catalan数

$$C(n) = \frac{1}{n+1} \binom{2n}{n} = \Omega\left(\frac{4^n}{n^{3/2}}\right) \quad \text{呈指数增长}$$

因此，穷举法不是一个有效算法。



矩阵连乘问题

◆动态规划

将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 简记为 $A[i:j]$ ，这里 $i \leq j$ 。

考察计算 $A[i:j]$ 的最优计算次序。设这个计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开， $i \leq k < j$ ，则其相应完全加括号方式为 $(A_i A_{i+1} \dots A_k)(A_{k+1} A_{k+2} \dots A_j)$ 。

→ **$A[i:j]$ 的计算量：** $A[i:k]$ 的计算量加上 $A[k+1:j]$ 的计算量，再加上 $A[i:k]$ 和 $A[k+1:j]$ 相乘的计算量



1. 分析最优解的结构

- 特征：计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的。
(证明?)

矩阵链乘的子问题空间： $A[i:j]$, $1 \leq i \leq j \leq n$

$A[1:1]$, $A[1:2]$, $A[1:3]$, ... , $A[1:n]$

$A[2:2]$, $A[2:3]$, ... , $A[2:n]$

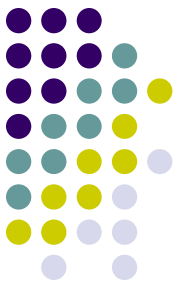
...

...

...

$A[n-1:n-1]$, $A[n-1:n]$

$A[n:n]$



2. 建立递归关系

- 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少乘次数 $m[i,j]$, 则原问题的最优值为 $m[1,n]$

取得的 k 为 $A[i:j]$ 最优次序中的断开位置, 并记录到表 $s[i][j]$ 中, 即 $s[i][j] \leftarrow k$ 。

注: $m[i][j]$ 实际是子问题最优解的解值, 保存下来避免重复计算。

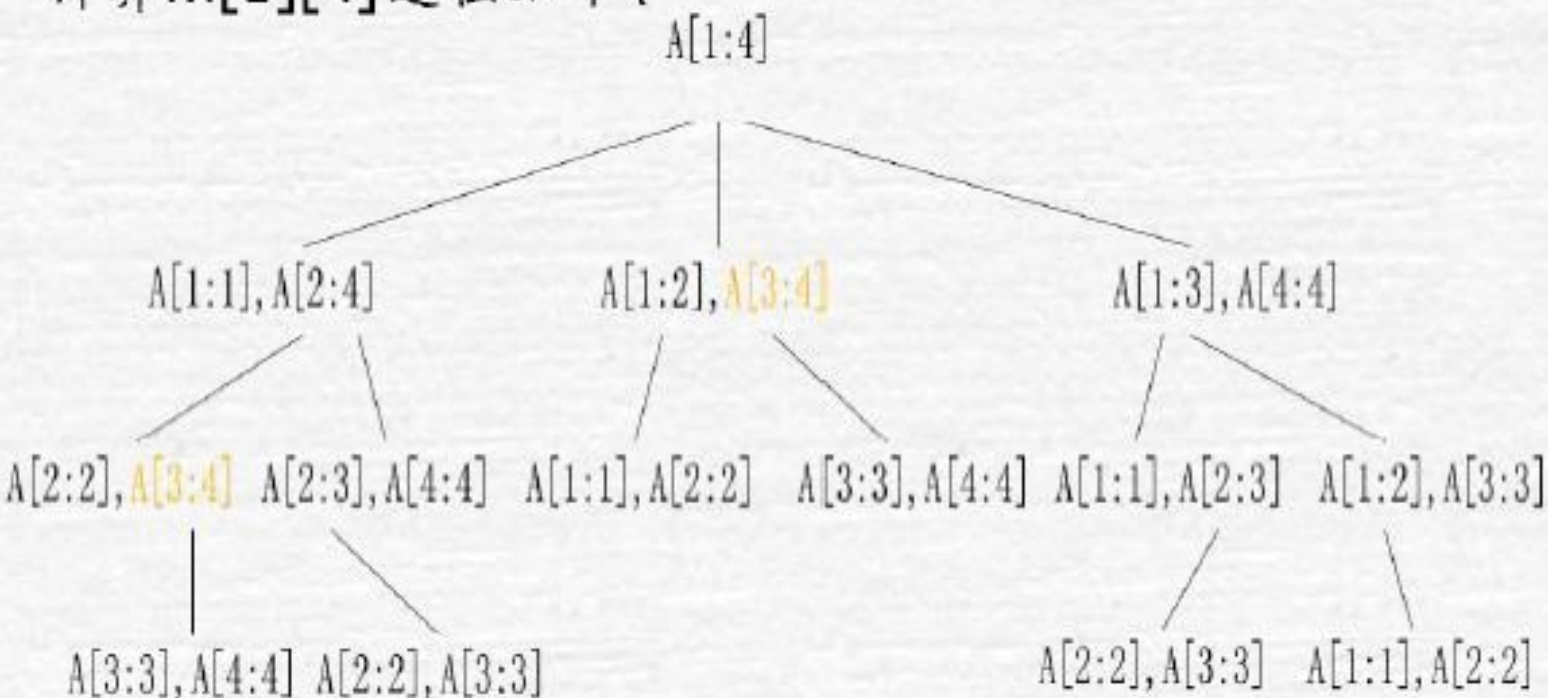
$$m[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

这里,

$$(A_i A_{i+1} \cdots A_k)_{p_{i-1} \times p_k} \times (A_{k+1} A_{k+2} \cdots A_j)_{p_k \times p_j}$$

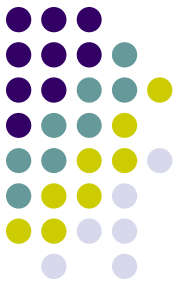


计算 $m[1][4]$ 过程如下:



如 $A[3:4]$ 被计算了2次, 保存下来可以节省许多时间

- 在递归计算时, 许多子问题被重复计算多次。这也是该问题可用动态规划算法求解的又一显著特征。



3. 计算最优值

```
void MatrixChain(int *p, int n, int **m, int **s) {  
    for (int r = 2; r <= n; r++)  
        for (int i = 1; i <= n-r+1; i++)  
            int j=i+r-1;  
            m[i][j] = m[i+1][j];  
            s[i][j] = i;  
            for (int k = i+1; k < j; k++) {  
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];  
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k;}  
            }  
        }  
    }  
}
```

***p** 存放 A_i 的维数

****m** 存放最优值

****s** 存放断开位置

3. 计算最优值



例 计算矩阵连乘积 $A_1A_2A_3A_4A_5A_6$

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$

		j					
		1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

		j					
		1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

3. 计算最优值



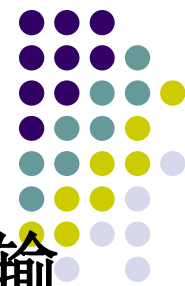
算法matrixChain复杂度分析：

算法的主要计算量取决于算法中对 r ， i 和 k 的3重循环。循环体内的计算量为 $O(1)$ ，而3重循环的总次数为 $O(n^3)$ 。

因此，算法的计算时间上界为 $O(n^3)$ 。

算法所占用的空间为 $O(n^2)$ 。

4. 构造最优解



按算法计算出的断点矩阵s指示的加括号方式输出计算A[i:j]的最优先次序。

```
void Traceback(int i, int j, int **s) {  
    if (i==j) return;  
    Traceback(i, s[i][j], s);  
    Traceback(s[i][j]+1, j, s);  
    cout<<"Multiplay A"<<i<<" ,"<<s[i][j];  
    cout<<"and A"<<s[i][j]+1<<" ,"<<j<<endl;  
}
```

动态规划算法的基本要素

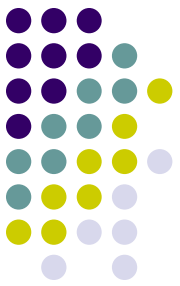


1. 最优子结构

- 如果问题的最优解是由其子问题的最优解来构造的，则称该问题具有最优子结构性质。
- 在分析问题的最优子结构性质时，通常是先假设由问题的最优解导出子问题的解不是最优的，然后再设法说明，在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。
- 利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。
- 最优子结构是使用动态规划算法求解的前提。

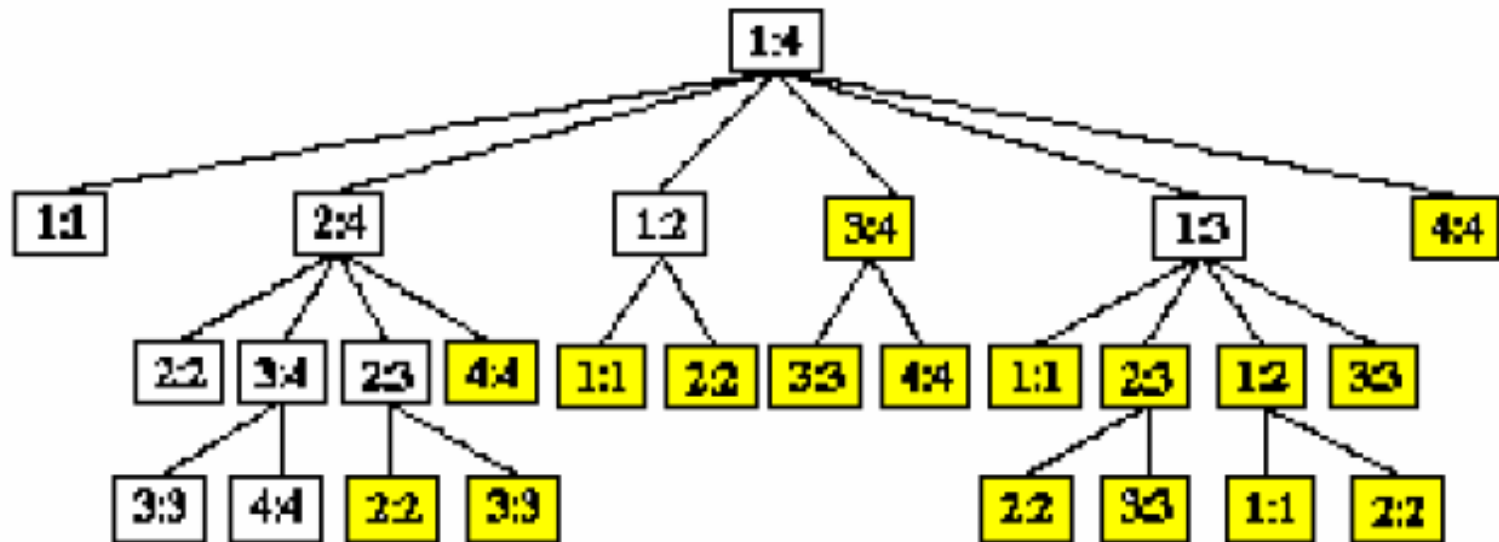
同一个问题可以有多种方式刻画它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）

动态规划算法的基本要素

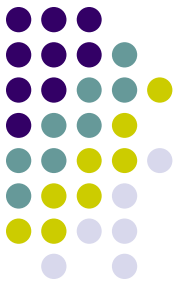


2. 重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**子问题的重叠性质**。



动态规划算法的基本要素



3. 备忘录方法

- 备忘录方法的控制结构与直接递归方法的控制结构相同，区别在于备忘录方法为每个解过的子问题建立了备忘录以备需要时查看，避免了相同子问题的重复求解。

```
int LookupChain(int i, int j) {  
    if (m[i][j] > 0) return m[i][j];  
    if (i == j) return 0;  
    int u = LookupChain(i, i) + LookupChain(i+1, j) + p[i-1]*p[i]*p[j];  
    s[i][j] = i;  
    for (int k = i+1; k < j; k++) {  
        int t = LookupChain(i, k) + LookupChain(k+1, j) + p[i-1]*p[k]*p[j];  
        if (t < u) { u = t; s[i][j] = k;}  
    }  
    m[i][j] = u;  
    return u;  
}
```



- 动态规划的设计技巧：**阶段的划分、状态的表示和存储表的设计**；
- 在动态规划的设计过程中，**阶段的划分和状态的表示**是其中最重要的两步，这两步会直接影响该问题的计算复杂性和存储表设计，有时候阶段划分或状态表示的不合理还会使得动态规划法不适用。
- 问题的阶段划分和状态表示，需要具体问题具体分析，没有一个清晰明朗的方法；
- 在实际应用中，许多问题的阶段划分并不明显，这时如果刻意地划分阶段法反而麻烦。一般来说，只要该问题可以划分成规模更小的子问题，并且原问题的最优解中包含了子问题的最优解（即满足**最优性原理**），则可以考虑用动态规划解决。



- 如何判断问题满足最优性原理?

思路：利用反证法，通过假设每一个子问题的解都不是最优解，然后导出矛盾，即可做到这一点。

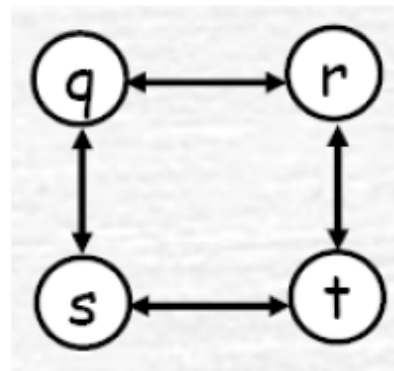
- 例1：设 G 是一个有向加权图，则 G 从顶点 i 到顶点 j 之间的最短路径问题满足最优性原理。



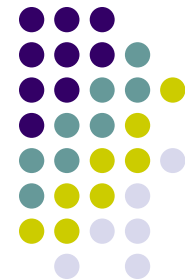
例2：有向图的最长路径问题不满足最优性原理。

证明：

如右图所示， $q \rightarrow r \rightarrow t$ 是 q 到 t 的最长路径，
而 $q \rightarrow r$ 的最长路径是 $q \rightarrow s \rightarrow t \rightarrow r$ ，
 $r \rightarrow t$ 的最长路径是 $r \rightarrow q \rightarrow s \rightarrow t$ ，
但 $q \rightarrow r$ 和 $r \rightarrow t$ 的最长路径合起来并不是 q 到 t 的最长路径。
所以，原问题并不满足最优性原理。



注：因为 $q \rightarrow r$ 和 $r \rightarrow t$ 的子问题都共享路径 $s \rightarrow t$ ，组合成原问题解时，有重复的路径对原问题是不允许的。



多段图规划

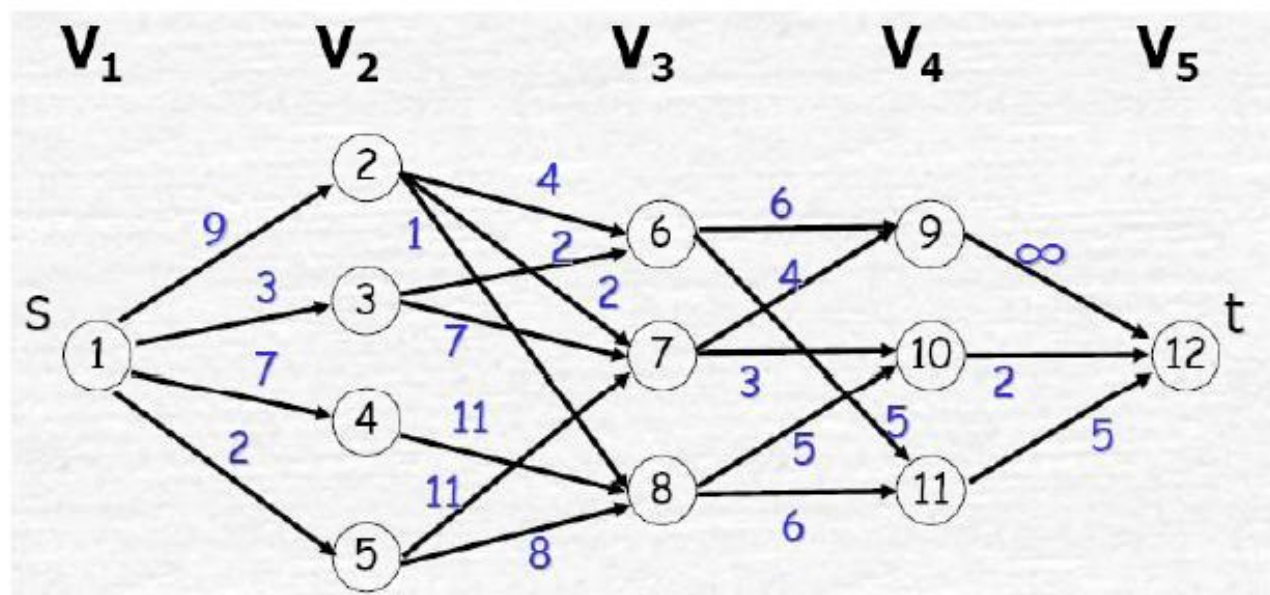
多段图规划



● 问题描述

多段图 $G=(V,E)$ 是一个有向图，且具有以下特征：

- (1) 划分为 $k \geq 2$ 个不相交的集合 $V_i, 1 \leq i \leq k$;
- (2) V_1 和 V_k 分别只有一个结点 s (源点)和 t (汇点);
- (3) 若 $\langle u, v \rangle \in E(G)$, $u \in V_i$, 则 $v \in V_{i+1}$, 边上成本记 $c(u, v)$;
若 $\langle u, v \rangle$ 不属于 $E(G)$, 边上成本记 $c(u, v) = \infty$ 。



求由 s 到 t 的最小成本路径。

多段图规划



- 多段图问题满足最优性原理

设 $s, \dots, v_{ip}, \dots, v_{iq}, \dots, t$ 是一条由 s 到 t 的最短路径, 则 $v_{ip}, \dots, v_{iq}, \dots, t$ 也是由 v_{ip} 到 t 的最短路径。(反证即可)

- 递归式推导

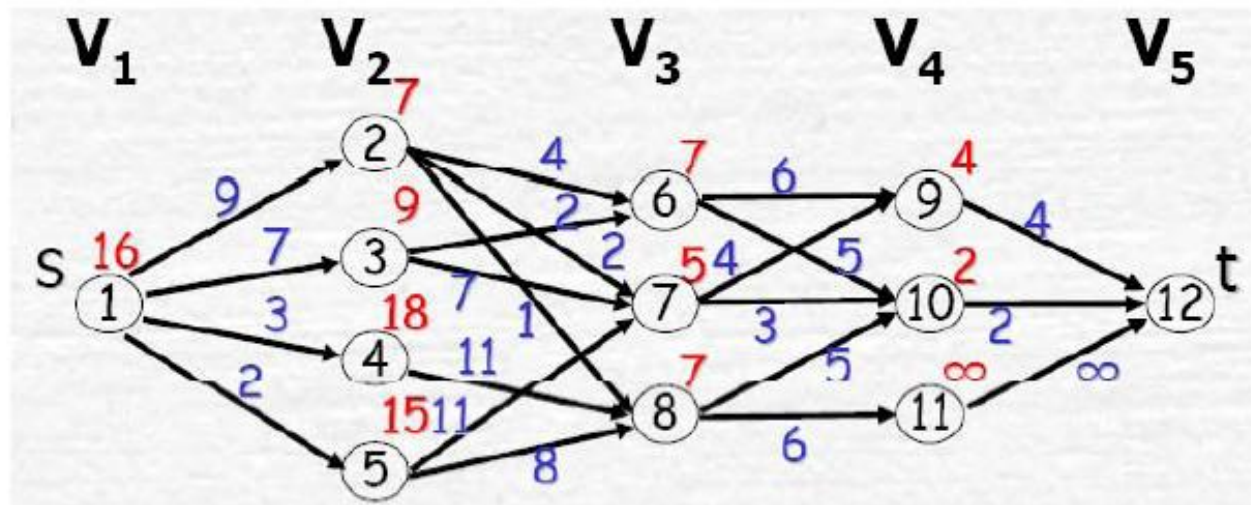
设 $\text{cost}(i, j)$ 是 V_i 中结点 v_j 到汇点 t 的最小成本路径的成本, 递归式为:

$$\text{cost}(i, j) = \begin{cases} c(j, t) & i = k - 1 \\ \min_{\substack{v_l \in V_{i+1} \\ \langle j, l \rangle \in E(G)}} \{c(j, l) + \text{cost}(i + 1, l)\} & 1 \leq i < k - 1 \end{cases}$$

多段图规划



● 计算过程(以5-段图为例)



$$\text{cost}(4, 9)=4, \text{cost}(4, 10)=2, \text{cost}(4, 11)=\infty$$

$$\text{cost}(3, 6)=7, \text{cost}(3, 7)=5, \text{cost}(3, 8)=7$$

$$\text{cost}(2, 2)=7, \text{cost}(2, 3)=9, \text{cost}(2, 4)=18, \text{cost}(2, 5)=15$$

$$\text{cost}(1, 1)=\min\{9+\text{cost}(2,2), 7+\text{cost}(2,3), 3+\text{cost}(2,4), 2+\text{cost}(2,5)\} = 16$$

构造解: 解1(1, 2, 7, 10, 12), 解2(1, 3, 6, 10, 12)

多段图规划



MultiStageGraph($G, k, n, p[]$)

{//输入 n 个结点的 k 段图, 假设顶点按段的顺序编号

// $E(G)$ 是边集, $p[1..k]$ 是最小成本路径

new cost[n]; //生成数组cost, cost[j]相当于前面的cost(i,j)

new d[n]; //生成数组d, d[j]保存 v_j 与下一阶段的最优连接点

cost[n]=0;

for i=n-1 dwonto 1 do //计算cost[i]和d[i]

{ cost[i]= ∞ ;

while(任意 $\langle i, r \rangle \in E(G)$) //r是下一阶段中的顶点

if($c(i, r) + \text{cost}[r] < \text{cost}[i]$)

{ cost[i]= $c(i, r) + \text{cost}[r]$; d[i]=r;

}

}

p[1]=1; p[k]=n; //以下是找一条最小成本路径 (构造解)

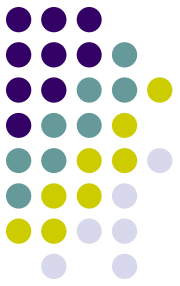
for i=2 to k-1 do p[i]=d[p[i-1]];

}

$\therefore T(n) = O(n+e)$

} $O(k)$

$O(n+e)$



本周任务

- 3-4
- 3-8
- 设计一个分治算法，判定两个二叉树T1和T2是否相同。
- 附加题
- 3-10
- 3-17